



浙江工业大学

本科生毕业论文（设计）

题目： 基于 Kubernetes 的一体化
DevOps 云平台系统设计

学 院： 管理学院

专 业： 信息管理与信息系统

班 级：

学 号：

学生姓名：

指导老师：

提交日期： 2024 年 6 月

浙江工业大学

本科生毕业设计(论文、创作)诚信承诺书

本人慎重承诺和声明：

- 1.本人在毕业设计（论文、创作）撰写过程中，严格遵守学校有关规定，恪守学术规范，所呈交的毕业设计（论文、创作）是在指导教师指导下独立完成的。
- 2.毕业设计（论文、创作）中无抄袭、剽窃或不正当引用他人学术观点、思想和学术成果，无虚构、篡改试验结果、统计资料、伪造数据和运算程序等情况。
- 3.若有违反学术纪律的行为，本人愿意承担一切责任，并且接受学校按有关规定给予的处理。

学生（签名）：

2025 年 4 月 19 日

基于 Kubernetes 的一体化 DevOps 云平台系统设计

摘 要

在当今数字化时代，软件行业发展迅猛，企业对软件产品的交付速度、质量和迭代能力要求极高。传统软件开发和部署模式问题重重，DevOps 理念应运而生，它强调开发与运维深度融合，借助自动化流程实现软件快速交付与持续改进。Kubernetes 作为容器编排领域的领先技术，能高效管理容器化应用，为解决大规模容器管理难题提供有效方案。本毕业设计聚焦于基于 Kubernetes 的一体化 DevOps 云平台设计，旨在使用 Go 语言将 Kubernetes 技术融入 DevOps 流程，搭建持续集成、持续交付、一键发布、日志查询、线上监报告警等功能于一体的云平台。通过对相关技术的研究和应用，实现开发部署流程标准化、提高交付效率、降低运维成本、增强系统稳定性与可扩展性。同时，对开发过程中的知识点进行归纳总结，以白皮书形式开源在 GitHub 网站，助力学生更好地掌握和运用计算机知识，模拟企业真实开发部署流程。本研究不仅有助于提升企业软件开发和部署效率，还为计算机相关专业学生提供了实践参考，对推动 DevOps 和 Kubernetes 技术的教育普及具有积极意义。

关键词：Kubernetes、Docker、DevOps、Go、SaaS

Design of an Integrated DevOps Cloud Platform System Based on Kubernetes

ABSTRACT

In today's digital age, the software industry is developing rapidly. Enterprises have extremely high requirements for the delivery speed, quality, and iteration ability of software products. The traditional software development and deployment model has many problems. The DevOps concept emerged as the times require. It emphasizes the deep integration of development and operation and maintenance, and realizes the rapid delivery and continuous improvement of software through automated processes. As a leading technology in the field of container orchestration, Kubernetes can efficiently manage containerized applications and provide an effective solution to the problem of large - scale container management. This graduation project focuses on the design of an integrated DevOps cloud platform based on Kubernetes. The aim is to integrate Kubernetes technology into the DevOps process and build a cloud platform that integrates functions such as continuous integration, continuous delivery, one - click release, one - click upgrade, log query, and online monitoring and alarming. Through the research and application of relevant technologies, the development and deployment process is standardized, the delivery efficiency is improved, the operation and maintenance cost is reduced, and the system stability and scalability are enhanced. At the same time, the knowledge points in the development process are summarized and archived, and a white paper is open - sourced on the GitHub website to help students better master and apply computer knowledge and simulate the real - world development and deployment process of enterprises. This research not only helps to improve the software development and deployment efficiency of enterprises, but also provides

practical reference for students majoring in computer - related fields, and has positive significance for promoting the education and popularization of DevOps and Kubernetes technologies.

KEY WORDS: Kubernetes、 Docker、 DevOps、 Go、 SaaS

目 录

摘 要	I
ABSTRACT	II
目 录	IV
图、表目录	VII
第 1 章 绪论	1
1.1 选题背景与意义	1
1.1.1 选题的背景	1
1.1.2 选题的意义	2
1.2 国内外研究现状	2
1.2.1 国内研究现状	2
1.2.2 国外研究现状	4
第 2 章 相关技术概述	6
2.1 Gin 框架	6
2.2 React	6
2.3 EFK	7
2.4 Docker	7
2.5 Kubernetes	8
2.6 Kafka	8
2.7 Redis	9
2.8 OSS	9
2.9 MySQL	9
2.10 WebSocket	10
2.11 CI/CD	10

第 3 章 系统架构设计	11
3.1 系统整体架构	11
3.1.1 表示层	12
3.1.2 网管层	12
3.1.3 应用层	13
3.1.4 监控层	16
3.1.5 基建层	17
3.1.6 第三方服务	18
3.2 数据库设计	19
3.2.1 数据库概念结构设计	19
3.2.2 数据库逻辑结构设计	26
第 4 章 功能需求分析	28
4.1 登录注册功能	28
4.2 Auth 鉴权模块功能	30
4.3 个人信息管理功能	31
4.4 GitHub 账号绑定功能	32
4.5 团队管理功能	33
4.6 团队申请功能	34
4.7 代码仓库渲染功能	35
4.8 Docker 镜像流水线构建功能	35
4.9 Kubernetes 资源信息管理功能	37
4.10 Kubernetes 资源配置文件保存和更新功能	38
4.11 Kubernetes 资源配置文件渲染功能	39
4.12 Kubernetes 资源操作功能	40
4.13 Kubernetes 资源定时监控功能	41
4.14 EFK 日志搜集功能	43
4.15 监控告警功能	43

第 5 章 系统实现	45
5.1 系统开发环境	45
5.2 系统功能实现	46
5.2.1 用户注册功能	46
5.2.2 用户登录功能	47
5.2.3 系统功能页面展示	48
5.2.4 GitHub 账号管理	51
5.2.5 Docker 账号管理	52
5.2.6 OSS 账号管理	53
5.2.7 团队创建	54
5.2.8 团队查询和团队申请	55
5.2.9 代码仓库管理	58
5.2.10 镜像文件管理	59
5.2.11 Docker 镜像构建	61
5.2.12 Kubernetes 资源信息管理	63
5.2.13 Kubernetes 资源操作	66
5.2.14 EFK 日志搜集	70
5.2.15 监控告警	71
第 6 章 总结与展望	72
6.1 总结	72
6.2 展望	72
参考文献	74
致 谢	76

图、表目录

图 3.1	项目架构图	11
图 3.2	用户实体	19
图 3.3	用户 OSS 实体	20
图 3.4	用户 Docker 账号实体	20
图 3.5	用户 GitHub 实体	21
图 3.6	团队实体	22
图 3.7	团队请求实体	22
图 3.8	Dockerfile 文件实体	23
图 3.9	Docker 镜像记录实体	23
图 3.10	Kubernetes 资源实体	24
图 3.11	Kubernetes 操作记录实体	25
图 3.12	实体逻辑结构图	26
图 4.1	用户登录时序图	29
图 4.2	用户注册时序图	30
图 4.3	Auth 鉴权流程图	31
图 4.4	个人信息用例图	32
图 4.5	GitHub 绑定时序图	33
图 4.6	团队信息管理用例	34
图 4.7	团队申请流程图	34
图 4.8	代码仓页面渲染时序图	35
图 4.9	镜像构建流程时序图	36
图 4.10	Kubernetes 部署用例图	37

图 4.11	Kubernetes 资源配置文件保存时序图	38
图 4.12	Kubernetes 资源配置文件更新时序图	39
图 4.13	Kubernetes 资源配置文件渲染时序图	40
图 4.14	Kubernetes 资源操作流程图	41
图 4.15	Kubernetes 资源定时监控时序图	42
图 4.16	EFK 数据流转图	43
图 4.17	监控告警流程图	44
图 5.1	注册页面	46
图 5.2	验证码邮件	46
图 5.3	验证码校验	47
图 5.4	登录页面	47
图 5.5	系统首页	48
图 5.6	个人中心	49
图 5.7	团队空间	49
图 5.8	代码仓库管理	50
图 5.9	镜像管理	50
图 5.10	Kubernetes 部署	51
图 5.11	GitHub 账户绑定	52
图 5.12	GitHub 账号解绑	52
图 5.13	Docker 账号查询	52
图 5.14	添加 Docker 账号	53
图 5.15	OSS 账号信息展示	53
图 5.16	OSS 账号绑定	54
图 5.17	团队创建	54
图 5.18	团队简介-1	55
图 5.19	团队查询	55
图 5.20	团队申请加入邮件	56

图 5.21	团队入队审批	56
图 5.22	团队审批结果邮件	56
图 5.23	团队简介-2	57
图 5.24	离队申请邮件	57
图 5.25	离队审批结果邮件	58
图 5.26	代码仓库管理详情	58
图 5.27	镜像文件管理	59
图 5.28	创建 Dockerfile	60
图 5.29	Dockerfile 文件导入	60
图 5.30	Dockerfile 详情查看	61
图 5.31	镜像构建-1	61
图 5.32	镜像构建-2	62
图 5.33	镜像构建-3	63
图 5.34	Kubernetes 部署详情	64
图 5.35	创建配置文件	65
图 5.36	OSS 查看文件	65
图 5.37	Kubernetes 资源操作记录	66
图 5.38	Kubernetes 资源部署	67
图 5.39	查看所有 Pod	67
图 5.40	查看所有 Service	67
图 5.41	查看所有 Deployment	68
图 5.42	查看所有 Namespace	68
图 5.43	查看集群信息	68
图 5.44	查看节点状态	68
图 5.45	运行中的资源	69
图 5.46	运行资源的操作	69
图 5.47	运行资源 Get 操作	69

图 5.48 运行资源 Describe 操作	69
图 5.49 Elasticsearch 动态索引	70
图 5.50 Elasticsearch 数据存储格式	71
图 5.51 容器监控告警	71
表 4.1 开发环境	45

第1章 绪论

1.1 选题背景与意义

1.1.1 选题的背景

在当今数字化时代，软件行业发展迅猛，市场竞争日益激烈，企业对软件产品的交付速度、质量以及迭代能力提出了极高要求。在软件开发的过程中，公司通常会将团队拆分为开发团队、测试团队和运维团队。这样的拆分是为了分解职能，各个团队可以各司其职，专精于自己分内的工作。在瀑布模式(Waterfall model)盛行的时候，一个项目的完成被拆分为 7 个阶段，分别是需求评估(Evaluation)、需求提出(Requirements)、需求分析(Analysis)、产品设计(Design)、系统开发(Development)、系统测试(Validation)、部署上线(Deployment)^[1]。虽然在瀑布模式下流程清晰，团队职能耦合低，团队更加专注于自己的任务。但是坏处也很明显，那就是一个项目依赖问题严重，流程容易阻塞，开发团队和运维团队沟壑严重。在此背景下，DevOps 应运而生。

DevOps 作为一种新兴的软件开发理念和实践方法，强调开发与运维的深度融合，通过自动化流程实现软件的快速交付和持续改进，有效缩短了软件从开发到上线的周期，提升了软件质量和可靠性。最终的结果是开发、运维、测试一体化，小需求迅速完成开发任务并部署上线，并周期性呈现开发、测试、部署三个过程^[2]。

同时随着云生态的不断完善，云设施的不断扩充，越来越多的公司将系统部署在云上。为此基于云原生的 DevOps 系统不断涌现。如今众多企业已采用 DevOps 并取得显著成效，如阿里的云效平台等，为软件的快速迭代和高质量开发提供了有力保障。然而，随着企业业务规模的不断扩大和应用场景的日益复杂，如何高效管理大量的容器化应用成为新的挑战。

Kubernetes 作为容器编排领域闪耀的明星，其由 Google 公司开发并完成^[3]。Kubernetes 能够自动化地部署、扩展和管理容器化应用程序，提供了强大的资源调度、服务发现和负载均衡等功能^[4]。它可以将多个容器组织成一个有机的整体，实现对容器集群的高效管理，确保应用在不同环境中的一致性和可靠性。将 Kubernetes 与 DevOps 相结合，构建一体化的云平台系统，成为企业提升软件开发和部署效率、增强竞争力的关键选择^[5]。目前，许多企业已经开始尝试将

Kubernetes 融入 DevOps 流程中，以实现更高效的软件交付和运维管理。

因此如何将 Kubernetes 技术融入到 DevOps 流程和系统平台中，简化项目部署流程，加快项目迭代速度，成为了当今研究的热点。同时我本人在实习过程中深度参与项目开发，深切感受到 DevOps 理念对开发和运维过程的重要影响：在多人开发工作流中公用一套测试环境和生产环境，项目代码都需要到测试环境进行测试校验，一天下来项目在测试环境的部署次数高达 10 次以上，如果没有合理的 DevOps 流程和运维系统将对我们的工作产生严重的阻碍。因此我才对基于 Kubernetes 的效能管理产生了浓厚的兴趣和技术热情。通过 Kubernetes 合理对容器进行编排和调度，并提供用户友好的 Web 界面进行快速管理成为了我毕业设计的首要目标。

1.1.2 选题的意义

如今 Kubernetes 成为了云原生开发的重要组件并已经被众多互联网公司采用，但是我们学生却对这一流行技术了解甚少，并对于企业内真正的 DevOps 开发部署流程处于一个模糊的状态。同时受制于 Kubernetes 和 Docker 等容器技术的晦涩难懂，一个开箱即用的 DevOps 运维管理平台便有了其存在的价值和意义。同学们只用通过简单的项目配置便可以搭建自己的 DevOps 工作流并对独自完成项目的开发和部署任务。

为此我希望通过我的毕业设计让更多的学生了解并真实模拟项目的开发和部署流程，让项目的运行不再停留在自己的个人电脑，让项目的启动不再是对 IDE（Integrated Development Environment）界面的手动操作，让项目的开发部署流程与企业内真实的工作流保持一致。

此外我将对我毕业设计中的知识点进行归纳和总结，知识点的梳理总结和项目源码将以白皮书的形式开源在 GitHub 网站上。通过开源和白皮书的形式，原本晦涩难懂的知识将跃然于纸上，计算机网络、数据库、操作系统的知识不再停留在书本而是落实于开发细节当中，帮助学生们更好的掌握并运用知识。

1.2 国内外研究现状

1.2.1 国内研究现状

在国内，随着数字化转型的加速推进，各行业对于高效软件开发与部署的需求极为迫切，这促使 Kubernetes 与 DevOps 相关研究和实践呈现出迅猛发展的态

势。

从技术应用层面来看，Kubernetes 在容器编排领域占据了主导地位。根据相关调查，早在 2019 年，CNCF 中国云原生调查就显示 72% 的受访者在生产中使用 Kubernetes，相较于 2018 年 11 月的 40% 有大幅提升^[6]。到如今，这一比例可能更为可观。众多企业，如京东、中国民生银行、蚂蚁金服等，都积极投身于 Kubernetes 的应用实践中。京东借助 Harbor (CNCF 项目) 作为镜像中央存储器，节省了约 60% 的维护时间^[7]；中国民生银行运用 Kubernetes 后，交付效率提高 3-4 倍，资源利用率实现翻倍^[8]；蚂蚁金服采用云原生技术（Kubernetes 是关键支撑），运营方面至少提升十倍^[9]。这充分表明 Kubernetes 能够有效助力企业提升资源管理效率，加速应用交付进程。

在 DevOps 企业级应用方面，国内也取得了显著进展。以 Gitee DevOps 平台为代表，它作为一站式研发效能平台，深度融合了代码托管、CI/CD、项目管理和安全审计等能力，贯穿软件开发从需求设计到生产运维的全生命周期。其具备本土化服务优势以及自主可控架构，严格遵循《数据安全法》与行业监管要求，支持私有化部署、代码加密及细粒度权限管理，成为金融、政务等对数据安全和合规性要求极高行业的首选。同时，Gitee 还能无缝对接企业微信、飞书等国内主流办公平台，并兼容 Jenkins、Ansible 等开源工具，降低了企业技术栈迁移的门槛。

此外根据研究显示^[10]18% 的受访者已将自动化 DevOps 组件（组件只是完整 DevOps 流程涉及到的中间件，并不是完成的系统平台）应用起来，同时有 43% 的受访者表示在未来的 6~24 个月内采用自动化 DevOps 组件。由此可见基于 Kubernetes 的运维管理平台正在蓬勃发展。

在学术研究领域，国内高校和科研机构针对 Kubernetes 和 DevOps 展开了大量研究。部分研究聚焦于如何优化 Kubernetes 集群的资源调度策略和调度算法，以提升资源利用率并降低成本。如杜军在研究中就首次提出了 Kubernetes 资源调度器资源抢占式调度设计、浮动式资源配额配置方案和中央调度器高可用解决方案^[11]。还有单朋荣等人使用 Prometheus 监控系统来自定义指标与采集业务指标进而实现多维度组合的最佳弹性伸缩方法^[12]。杨茂、陈莉君创新提出了预测式伸缩与响应式伸缩相结合的伸缩算法，从而有效改善响应延迟问题，降低响应时间，提高应用服务质量^[13]。还有一些研究关注 DevOps 流程中的自动化测试与持续集成优化，如胡鹏等人通过定义用例属性标签、动态选择测试用例、自动排序用例优先级以及执行中动态选择测试用例等方法，实现回归测试的持续优化，以降低执行用例规模、提高缺陷检测效率^[14]。

不过，国内在 Kubernetes 和 DevOps 的发展过程中也面临一些挑战。例如，

容器化应用的安全问题日益凸显，尽管有相关安全措施，但如 Tenable Cloud Research 发布的研究成果显示^[15]，全球范围内包括亚太地区企业的 Kubernetes 环境都存在安全隐患，国内企业也难以独善其身，像公开暴露的 API 服务器、权限过高的容器等问题都亟待解决。同时，不同企业在 DevOps 实践水平上参差不齐，部分中小企业由于缺乏专业技术人才和足够的资金投入，在构建和完善 DevOps 体系时面临诸多困难。

1.2.2 国外研究现状

国外对于 Kubernetes 和 DevOps 的研究与实践起步较早，在技术创新、应用深度和广度等方面都处于领先地位。

Kubernetes 自诞生以来，在全球范围内得到广泛应用和持续创新。据云原生计算基金会（CNCF）报告，Kubernetes 是全球第二大开源项目（仅次于 Linux），并且是 71% 的财富 100 强公司的主要容器编排工具^[16]。它最初由 Google 开发，作为 Borg 的开源版本于 2014 年亮相，2015 年被捐献给 CNCF 后，迅速成为发展最快的开源项目之一，并在 2017 年超越 Docker Swarm 和 Apache Mesos 等竞争对手，确立了在容器编排领域的行业标准地位^[17]。如今，Kubernetes 不断进化，其生态系统极为丰富，涵盖众多插件、工具和服务。例如，HashiCorp 的 Nomad 与 Kubernetes 形成竞争互补关系，为企业提供更多容器编排选择；而 Red Hat 的 OpenShift 基于 Kubernetes 构建，为企业提供了更全面、企业级的容器应用平台，简化了企业对 Kubernetes 的部署和管理工作。

在 DevOps 领域，国外企业长期处于实践前沿，形成了成熟的方法论和工具链体系。许多知名企业，如 Netflix、Amazon 等，通过 DevOps 实践实现了高效的软件交付和业务创新。Netflix 开源的 Spinnaker 专注于复杂多云环境的持续交付，支持多种部署策略，并结合可视化流水线设计，极大地降低了发布风险，提升了灰度验证效率^[18]。Amazon 则凭借自身强大的云计算服务 AWS，为企业提供了一整套完善的 DevOps 工具和服务，涵盖代码托管、构建、测试、部署以及监控等各个环节。此外，像 Pulumi 这样的工具突破传统 IaC（基础设施即代码）工具限制，支持 Python、TypeScript、Go 等通用编程语言定义云资源和执行策略，显著提升了代码可维护性，其状态管理与策略即代码（Policy as Code）功能，为跨云架构提供了安全、可复用的编排方案^[19]。

在研究方面，国外高校和科研机构对 Kubernetes 和 DevOps 的研究深入且广泛。在 Kubernetes 资源管理与调度优化上，有大量研究运用机器学习、人工智能

等方法，旨在进一步提升集群资源利用率 and 应用性能。例如 RubakAdam 等人^[20]使用多种机器学习方法预测用户数量增加后的资源需求，进而优化 Kubernetes 平台的 Pod 容量分配问题。Jorge-Martinez D 等人^[21]通过人工智能进行辅助决策，控制节点的调度和负载转移，进而改进用户数字化请求的容器调度，从而提高调度效率并降低成本。对于 DevOps，研究方向集中在如何让不同的工作人员更好地融合 DevOps 理念和实际流程中，如 LeiteLeonardo^[22]等人提出：对于工程师应主动学习 DevOps 技能，尽可能重新设计系统架构以支持 DevOps 并选择合适的工具集；对于管理者应该与工程师一起选择最佳的 DevOps 自动化工具集，且合理评估已采用的 DevOps 实践的质量；对于研究者应该在课程中引入 DevOps 原理与实践，从而培养新一代工程师。

然而，即便在国外，Kubernetes 和 DevOps 的发展也并非一帆风顺。Tenable 的《2024 年云风险报告》指出^[15]，随着容器化应用程序在各行业的广泛应用，安全措施却未能跟上步伐，致使云基础设施面临数据泄露、服务中断及关键工作负载被非法访问的高风险。报告显示^[15]78%的组织其 Kubernetes API 服务器可被公开访问，74%的组织有暴露的公开存储数据。这表明在 Kubernetes 技术快速发展的同时，安全问题仍是全球范围内亟待攻克的难题。

第2章 相关技术概述

毕设整体使用单体项目架构，后端使用 go 语言 Gin 框架，前端使用 typescript 语言 React 框架完成了整体的项目开发任务。项目涉及到了诸多第三方中间件和相关理论知识和技术，包括：MySQL、Redis、Kafka、EFK、Docker、Kubernetes、OSS、WebSocket、CI/CD。

2.1 Gin 框架

Gin 是一个用 Go 语言编写的轻量级 Web 框架，它基于 HTTP 协议，采用了高性能的路由引擎和中间件机制。Gin 的设计目标是提供一种简单、高效的方式来构建 Web 服务器和 API 接口。它具有极低的内存占用和高并发处理能力，能够快速响应用户的请求。Gin 框架提供了丰富、易于上手的网络组件功能，包括路由分发、网络参数绑定、错误处理等，使开发者快速开发和维护 Web 应用程序。

Gin 框架主要用于快速构建 Web 服务器和 API 接口。它可以处理各种类型的 HTTP 请求，如 GET、POST、PUT、DELETE 等，并根据请求的路径和参数进行相应的处理。通过使用 Gin 框架，开发者可以快速搭建起一个功能完善的 Web 应用程序，减少了开发时间和工作量。同时，Gin 框架的高性能和稳定性也使得它非常适合用于构建高并发的 Web 服务。

Gin 框架常用于构建 RESTful API、Web 应用程序等业务场景。

2.2 React

React 是一个用于构建用户界面的 JavaScript 库，采用组件化和虚拟 DOM 的开发方式。虚拟 DOM 是真实 DOM 的轻量级抽象表示，通过比较虚拟 DOM 差异，React 只更新必要的真实 DOM 节点，从而提升渲染性能。组件化开发将页面拆分为独立组件，每个组件管理自身的功能和样式，有效提高代码的可维护性和复用性。

React 用于构建 Web 应用程序的前端界面，实现用户交互和数据展示。通过使用 React，可以创建动态、交互性强的用户界面，提高用户体验。React 提供了丰富的组件库和工具，方便开发者快速构建页面。同时，React 还支持与其他前端框架和库的集成，如 Redux、React Router 等，可以进一步扩展应用程序

的功能。

2.3 EFK

EFK 不是一个软件，而是一套解决方案，开源软件之间的互相配合使用，高效的满足了很多场合的应用，是目前主流的一种日志系统。EFK 是三个开源软件的缩写，分别表示：Elasticsearch, Flutend, Kibana，其中 ELasticsearch 负责日志保存和搜索，Flutend 负责收集日志。

Elasticsearch 是一个基于 Lucene 构建的分布式搜索和分析引擎，采用 RESTful 风格，具备强大的全文搜索与数据分析能力。它支持存储海量数据，并提供实时搜索和分析功能。Elasticsearch 采用分布式架构，将数据分散存储于多个节点，提升系统的可扩展性和容错性。其核心数据结构——倒排索引，通过将文档中的词项映射到包含该词项的文档列表，与传统正向索引相反，从而实现高效的全文搜索。

Fluentd 一个开源的数据收集器，用于收集、处理和转发日志数据。Fluentd 可以从各种数据源（如文件、网络、数据库等）收集日志信息，并将其转换为统一的格式，然后转发到目标存储系统（如 Elasticsearch、Kafka 等）。它具有高度定制性和插件化的架构，可以根据不同的需求进行配置和扩展。

Kibana 一个可视化工具，用于展示 Elasticsearch 中的数据。Kibana 提供了丰富的可视化组件，如柱状图、折线图、饼图等，可以帮助开发者和运维人员直观地查看和分析日志数据。同时，Kibana 还支持创建仪表盘和报表，方便用户对数据进行深入分析和监控。

EFK 主要用于日志收集、存储和可视化，帮助开发人员和运维人员监控和分析应用程序的运行状态。通过收集和分析应用程序的日志信息，可以及时发现系统中的问题和异常，如错误日志、性能瓶颈等，并采取相应的措施进行处理。同时，EFK 还可以帮助用户了解系统的使用情况和用户行为，为系统的优化和改进提供依据。

2.4 Docker

Docker 是一个开源的容器化平台，它使用容器来打包应用程序及其依赖项^[23]。容器是一种轻量级的虚拟化技术，它将应用程序和其依赖的库、环境等封装在一个独立的单元中，实现了应用程序的隔离和快速部署。Docker 采用了分层存储和镜像技术，使得容器的创建和部署非常快速和高效。同时，Docker 还提

供了丰富的命令行工具和 API，方便用户对容器进行管理和操作。

Docker 的主要用途是简化应用程序的部署和管理。通过使用 Docker 容器，可以将应用程序及其依赖项打包成一个独立的镜像，然后在不同的环境中进行部署。这样可以避免因环境差异导致的应用程序无法正常运行问题，提高了开发、测试和生产环境的一致性。此外，Docker 还支持容器的快速启动和停止，使得应用程序的部署和更新更加方便快捷。

2.5 Kubernetes

Kubernetes 是一个开源的容器编排平台，用于容器镜像的自动化部署、动态扩容和自动管理容器化应用程序^[24]。Kubernetes 采用了分布式架构，由多个组件组成，包括控制平面和工作节点。控制平面负责管理和调度集群中的资源，如节点、容器、服务等；工作节点则负责运行容器化应用程序。Kubernetes 提供了丰富的功能，如自动伸缩、负载均衡、滚动更新、故障恢复等，使得容器化应用程序的管理更加简单和高效。

Kubernetes 的核心目标是实现容器自动化管理，提升应用程序的可用性和可扩展性。通过使用 Kubectl 命令可以轻松实现容器的自动部署、扩展和编排管理，减少了人工干预和错误。同时，Kubernetes 还支持多节点集群的管理，可以将应用程序分布在多个节点上运行，提高了系统的容错性和性能。此外，Kubernetes 还提供了丰富的插件和扩展机制，可以根据不同的需求进行定制和扩展。

2.6 Kafka

Kafka 是一个分布式流处理平台，它基于发布 - 订阅模式，用于处理大量的实时数据流。Kafka 采用了分布式架构，由多个 broker 组成。每个 broker 负责存储和处理部分生产者发送的数据。Kafka 将数据存储在主题（Topic）中，每个主题可以被分为多个分区（Partition），每个分区又可自定义多个副本（Replica）以保证数据的高可用性。生产者（Producer）将数据发布到主题中，由消费者（Consumer）从 Topic 中订阅并消费数据。Kafka 具有高吞吐量、低延迟、可扩展性等特点，可以处理每秒数百万条消息的流量。

Kafka 主要用于消息队列、数据采集、日志处理等场景，实现系统之间的解耦和异步通信。通过使用 Kafka，可以将不同的系统和组件之间的通信解耦，提高了系统的灵活性和可维护性。同时，Kafka 还支持消息的持久化存储和多副本机制，保证了数据的可靠性和可用性。此外，Kafka 还提供了丰富的 API 和工具，方便用户对消息进行生产、消费和管理。

2.7 Redis

Redis 是一个开源的内存数据存储系统，支持字符串、哈希、列表、集合等多种数据结构。数据存储在内存中，赋予其极高的读写性能，同时支持持久化存储，可定期或实时将数据保存至磁盘以防丢失。Redis 采用单线程模型，结合异步 I/O 和事件驱动机制，确保高并发处理能力。

Redis 广泛应用于缓存、会话管理、消息队列和分布式锁等场景，以提升系统性能和响应速度。作为缓存，Redis 可存储频繁访问的数据，减少数据库访问次数，加快响应速度；在会话管理中，可存储用户会话信息，实现分布式会话共享；作为消息队列，Redis 支持系统间的异步通信。

2.8 OSS

OSS 是一种基于云计算的存储服务，用于存储和管理大量的非结构化数据，如图片、视频、文档等。OSS 由于使用了分布式架构，数据依据冷-热访问情况分散存储在多个节点上，提高了数据的可靠性和可用性。OSS 提供了简单易用的 API 和工具，方便用户对数据进行上传、下载和管理。同时，OSS 还支持数据的冗余存储和备份，保证了数据的安全性和完整性。

OSS 提供可靠、安全、低成本的存储解决方案，方便用户上传、下载和管理数据。通过使用 OSS，用户可以将大量的非结构化数据存储在云端，无需自己搭建和维护存储系统，降低了存储成本和管理难度。

2.9 MySQL

MySQL 是一个开源的关系型数据库管理系统，采用客户端-服务器架构，由客户端和服务端组成。客户端通过 SQL 语句向服务器发送请求，服务器处理请求并返回结果。MySQL 支持多种数据类型和 SQL 语句，具有高性能、可靠性和易用性。它采用多线程、多用户设计，能同时处理多个客户端请求，提升并发能力。

MySQL 用于存储和管理结构化数据，如用户信息和业务数据，通过组织化的存储结构便于查询、插入、更新和删除操作。它支持事务处理和数据完整性约束，确保数据一致性和可靠性。此外，MySQL 提供丰富的管理工具和插件，方便数据库的维护和管理。

2.10 WebSocket

WebSocket 是基于 TCP 协议的全双工通信协议，它允许浏览器和服务端之间进行实时通信。WebSocket 协议在 HTTP 协议的基础上进行了扩展，通过握手机制建立连接，然后在连接上进行双向数据传输。WebSocket 协议支持二进制数据和文本数据的传输，具有低延迟、高吞吐量等特点。

WebSocket 用于实现实时通信、在线聊天、实时数据更新等场景。通过使用 WebSocket，可以在浏览器和服务端之间建立实时连接，实现数据的实时传输和更新。与传统的 HTTP 请求 - 响应模式相比，WebSocket 可以减少网络延迟，提高用户体验。同时，WebSocket 还支持多用户之间的实时通信，如在线聊天、多人游戏等。

2.11 CI/CD

CI 意为持续集成（Continuous Integration）。CD 意为持续交付 / 部署（Continuous Delivery/Deployment）。

持续集成（Continuous Integration）是指开发人员将代码合并到远程代码仓库中，每次合并后都会自动或手动触发项目构建和项目测试流程。这个过程会对代码进行编译、运行单元测试、集成测试等操作，以尽早发现代码冲突和缺陷。通过持续集成，团队成员能够及时了解代码的健康状况，确保新代码不会破坏现有的功能。例如，在一个多人开发的项目中，每天可能会有多次代码合并，每次合并后都会自动运行测试，一旦发现问题，开发人员可以立即进行修复。

持续交付（CD - Continuous Delivery）是在持续集成的基础上，确保所有通过测试的代码可以随时部署到生产环境。持续交付强调代码的可部署性，它会自动完成一系列的准备工作，如打包、配置、部署到预生产环境进行进一步的测试等。但最终是否将代码部署到生产环境，仍然需要人工手动触发。例如，经过测试的代码会被打包成可部署的版本，存放在一个待部署的仓库中，当业务需求允许时，由相关人员手动将其部署到生产环境。

持续部署（CD - Continuous Deployment）是持续交付的更高阶段，它将代码部署到生产环境的过程也自动化。一旦代码通过了所有的测试和检查，就会自动部署到生产环境中，无需人工干预。这使得软件的更新能够更快地推送给用户，提高了软件的交付速度和响应能力。例如，互联网产品在代码完成开发和测试后，会自动部署到生产服务器上，用户可以立即使用到新的功能。

第3章 系统架构设计

3.1 系统整体架构

本毕业设计基于 Docker 和 Kubernetes 两大核心技术，实现了完整的 CI/CD 流程。项目基于 Docker 的一键持续集成和持续交付流水线，并基于 Kubernetes 实现一键持续部署。此外并使用 EFK 和 Kakfa 技术实现 Kubernetes 运行资源的日志搜集和监控告警功能。下面是项目设计的完整架构，如图 3.1 项目架构图所示。

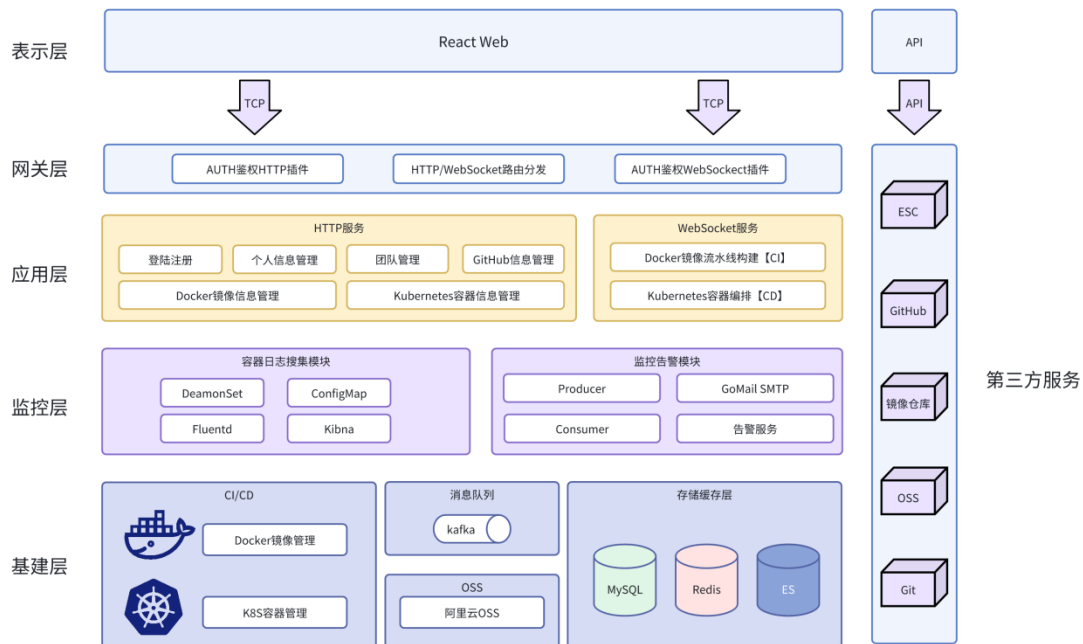


图 3.1 项目架构图

项目整体使用 B/S 前后端分离的架构设计，使用 Typescript 语言和 React 框架完成了完整的前端开发任务，使用 Go 语言和 Gin 框架完成了后端的开发任务。前后端均采用架构清晰、稳定的单体项目架构。

项目整体架构分为 6 层，分别为：前端用户交互的表示层、Auth 用户鉴权的网管层、核心功能模块的应用层、日志搜集和监控告警的监控层、核心中间件的基层层。

3.1.1 表示层

在本项目中，前端表示层的构建全面采用了 TypeScript 和 React 框架。TypeScript 作为 JavaScript 的超集，为代码注入了静态类型检查的特性，极大地增强了代码的可维护性和可扩展性，使得开发人员能够在编译阶段就捕获到许多潜在的错误。而 React 框架则以其高效的虚拟 DOM 机制和组件化开发模式，让页面的构建更加灵活和高效，能够快速响应业务需求的变化。

表示层的核心目标是为用户打造一系列贴合需求且页面精美的 Web 页面。这些页面就像是用户与系统之间的友好桥梁，提供了一站式的 CI/CD 操作工具、个人信息管理功能、团队信息管理功能。

表示层作为面向用户的重要窗口，承担着将后端接口功能进行可视化转换的重要任务。后端提供的接口功能通常是格式化、标准化的，但对于普通用户来说，这些接口信息并不直观。表示层通过将这些接口功能转换为各种可视化组件，如功能按钮、弹窗、列表展示等，让用户能够轻松地使用系统的各项功能。例如，用户在进行 CI/CD 操作时，只需点击相应的功能按钮，系统就会自动调用后端的 API 接口完成相应的任务。用户无需了解后端 API 接口的具体实现细节，只需关注页面上的可视化操作即可完成项目提供的核心功能。

3.1.2 网管层

网关层集成后端功能服务对外暴露的所有 API，为表示层提供可靠的网络连接和路由分发，并对部分 API 进行 Auth 鉴权从而保证后端功能服务安全稳定的运行。

在与表示层的交互方面，网关层整体采用 TCP 协议。TCP 协议以其可靠的连接机制和数据传输保证，确保了数据在传输过程中的稳定性和可靠性。无论是在网络状况良好还是存在一定干扰的情况下，TCP 协议都能够有效地保证数据的准确传输，避免数据丢失或损坏。现项目整体提供 HTTP 和 WebSocket 两种协议以保证完整地提供应用层服务能力。

对于后端暴露的大多数 API，使用 HTTP 协议进行数据交互。HTTP 协议是一种无状态的协议，简单易用，适用于大多数数据请求场景。它能够快速地将用户的请求发送到后端服务器，并接收服务器返回的响应数据。

而在 CI/CD 模块中，为了满足实时性和时效性的要求，采用了长连接的 WebSocket 协议。WebSocket 协议允许表示层和后端服务服务器之间建立持久的连接，实现双向实时通信，进而让用户能够及时掌握系统的运行状态。

对于路由分发和 Auth 鉴权，网关会拦截表示层请求中的路由地址，进而将请求分发给应用层不同的功能模块中以实现不同的业务逻辑。在路由分发的过程中，网关层会对部分路由进行 Auth 鉴权，只有请求头中携带 Authorization 字段的请求才被允许放行。在 Auth 鉴权的过程中，网关层通过 Authorization 字段信息从 Redis 获取缓存中的用户信息，并加载如请求上下文中。通过网关层的 Auth 鉴权，一方面拦截了非法的用户请求，另一方面保护了用户信息不被浏览器捕捉，保证了系统和用户信息的安全性。

3.1.3 应用层

应用层是后端项目的核心内容，一方面对外提供稳定且强大的 API 供白表示层与用户交互，另一方面对内与监控层和基建层深绑定提供 CI/CD、容器日志采集、监控告警的稳定服务。

应用层根据对外提供的 API 协议分为 HTTP 服务模块和 WebSocket 服务模块。HTTP 服务模块中主要的功能模块分别是：登录页注册模块、个人信息管理模块、GitHub 信息管理模块、团队管理模块、Docker 镜像管理模块、Kubernetes 信息管理模块。WebSocket 服务模块的主要功能模块是分别是两个：Docker 镜像流水线构建（CI 过程）和 Kubernetes 容器编排（CD 过程）。

（1）登录注册模块

登录注册模块是项目服务的入口，也是服务 Auth 鉴权的前提。用户根据个人邮箱和密码进行登录和注册校验。用户登录注册成功后又登录注册模块返回一个 24 小时有效的 token 对象，然后进入表示层使用本项目产品。

Token 对象将由表示层存储在用户使用浏览器的 local storage 中，允许表示层在任何时刻获取用户 token。在需要鉴权的 API 中，该 token 是 Auth 鉴权的唯一标识。任何无效、非法 token 请求都将被主动拦截，拒绝访问。

登录注册模块的稳定是项目正常运行的前提。

（2）GitHub 信息管理模块

GitHub 信息管理是项目服务合理运行的前提，整个项目运行的前提都是基于用户绑定的 GitHub 账号实现的。在 GitHub 信息管理模块中，用户需要在本系统中绑定 GitHub 账号，从而使系统有权限获取用户的代码仓库数据，进而在不同的业务场景下将不同的 GitHub 仓库数据经过统一处理后呈现在表示层为用户提供服务。

GitHub 信息管理模块服务负责绑定与解绑用户的 GitHub 账号。GitHub

信息管理模块通过 GitHub 官方提供的 API 实现用户信息的绑定与解绑操作。对于绑定 GitHub 账号的用户，系统将获取用户的 GitHub Access Token，进而通过 GitHub 提供的 API 获取用户的用户名、仓库信息、头像 URL、GitHub 注册邮箱等个人信息。用户 GitHub 账号信息将被持久化存入 MySQL 中，并在“代码仓库管理”、“Docker 镜像管理”、“Kubernetes 部署”三个业务模块合理使用：获取用户代码仓库元数据信息、代码文件、代码仓库分支信息、代码仓库提交信息等。

(3) 个人信息管理模块

个人信息管理模块负责维护用户的 Docker 账号管理和 OSS 账号管理。Docker 账号负责记录 Docker 镜像仓库地址、命名空间等 Docker 账号的关键信息。其作用在于持续交付的时候将构建成功的 Docker 镜像上传至镜像仓库中。OSS 账号管理负责维护用户的 OSS 账号信息，包括 Access Key ID、Access Key Secret、Bucket、Region 信息。OSS 账号信息主要用于保存和拉取 Kubernetes 资源文件时使用。表示层和 Kubernetes 容器编排模块需要 OSS 实现资源文件的上传和拉去，从而保存和获取文件信息。

(4) 团队管理模块

团队是在业务上引入的一层概念，旨在“Docker 镜像信息管理”和“Kubernetes 容器信息管理”中允许用户个人查看团队整体的 Docker 镜像资源和 Kubernetes 容器资源。为此开发团队管理模块用于维护团队信息和团队成员信息。

团队管理模块允许用户创建个人团队也允许加入任意一个团队。对于创建个人团队，用户自己将成为管理员，并拥有团队的管理权限，负责团队成员入队和离队的审批操作。而对于加入和离开团队，用户将通过点击在表示层的按钮完成入队和离队操作。入队和离队操作的审批都将通过邮件的形式发送给团队管理员，由团队管理员负责审批通过或审批拒绝。

(5) Docker 镜像信息管理模块

Docker 镜像信息管理是 Docker 镜像流水线构建(CI)的前置功能。Docker 镜像信息管理模块主要负责 Dockerfile 文件的保存、修改、删除、列表查询，以及 Docker 镜像构建日志的查询功能。对于 Docker 镜像构建使用的 Dockerfile 文件，Docker 镜像信息管理模块将以 JSON 的形式将用户定义的 Dockerfile 文件保存到 MySQL 数据库中，并提供相应的增加、修改、删除、列表查询接口与表示层交互。对于 Docker 镜像构建日志查询功能，在 Docker 镜像流水线构建完成后将日志记录数据写入 MySQL 数据库，并提供列表查

询接口供表示层使用。

(6) Kubernetes 容器信息管理

Kubernetes 容器信息管理模块是 Kubernetes 容器编排(CD)的前置功能，主要负责 Kubernetes 资源配置文件的增加、更新、删除、查看容器操作记录等功能。由于 Kubernetes 的容器运行与配置文件是强绑定关系，Kubernetes 容器资源的启动必须依赖一个配置文件，因此需要一个单独的模块用于维护 Kubernetes 的资源文件。其中对于资源文件的保存操作，Kubernetes 容器信息管理模块只负责存储文件的 OSS URL 地址，并不负责实体文件的保存操作。实体文件的保存由表示层上传到 OSS，从而减少网络流量的消耗，进而降低系统运行的成本。

(7) Docker 镜像流水线构建 (CI/CD) 模块

在本系统的整体架构中，Docker 镜像流水线构建模块承担着至关重要的职责，它主要负责对用户 GitHub 代码仓库进行 Docker 镜像构建任务，其是本系统的核心功能之一。这一模块的存在，极大地简化了从代码到可运行镜像的转化流程，为后续的容器部署和应用运行奠定了坚实基础，在整个软件开发和部署的生命周期中扮演着承上启下的关键角色。

Docker 镜像流水线构建主要细致地划分为 5 个步骤，分别是：Docker 账号登录、GitHub 代码仓库的克隆、Dockerfile 文件构建、构建 Docker 镜像、Docker 镜像推送。这 5 个步骤相互关联、环环相扣，以流水线的形式先后依次完成，确保整个构建过程有条不紊、高效推进。

在整个流水线构建期间，表示层将和应用层保持 WebSocket 长连接状态。WebSocket 长连接为表示层和应用层之间搭建了一座实时通信的桥梁，使得双方能够及时交换信息。应用层将不断地将流水线运行日志推送给表示层，这些日志详细记录了每一个步骤的执行情况，包括开始时间、结束时间、执行结果等。用户可以通过表示层直观地查看这些日志信息，从而感知到构建的执行流程。这种实时的日志反馈机制，大大提高了用户对构建过程的掌控能力，降低了构建失败的风险，确保了整个 Docker 镜像流水线构建过程的高效、稳定运行。

(8) Kubernetes 容器编排 (CD) 模块

Kubernetes 容器编排模块主要负责 Kubernetes 的资源管理，实现了持续部署 (CD) 功能，其是本系统的核心功能之一。

Kubernetes 容器编排模块为系统提供了丰富多样的 Kubectl 内置命令。这些命令就像是一套强大的工具箱，涵盖了 Kubernetes 资源管理和操作的

各个方面。表示层可以按照规定的输入格式，通过 WebSocket 消息将这些命令传递给 Kubernetes 节点。WebSocket 协议的使用确保了数据传输的实时性和稳定性，使得表示层和 Kubernetes 节点之间能够高效地进行通信。

在实际操作中，这些命令为用户提供了极大的便利。以 Kubernetes 资源一键部署功能为例，用户只需在表示层点击相应的按钮，系统就会自动将预先配置好的 Kubernetes 资源（如 Deployment、Service 等）部署到集群中。整个过程快速而准确，大大节省了用户的时间和精力。一键销毁功能则允许用户在需要时，迅速将不再使用的 Kubernetes 资源从集群中移除，释放资源空间。而一键查看容器状态和 Kubernetes 集群状态功能，使用户能够实时了解容器和集群的运行情况，及时发现并解决潜在的问题。例如，当容器出现异常时，用户可以通过查看容器状态信息，快速定位问题所在，并采取相应的措施进行修复。

强大且健壮的 Kubernetes 容器编排模块承担了持续部署的全部任务。它不仅要确保新的代码和功能能够顺利部署到生产环境中，还要保证部署过程的稳定性和可靠性。在部署过程中，模块会进行严格的检查和验证，确保新的部署不会对现有系统造成影响。

3.1.4 监控层

在本系统的整体架构中，监控层起着至关重要的作用，它就像系统的“眼睛”和“耳朵”，实时洞察系统的运行状况，及时发现潜在的问题和异常。监控层主要由两个关键部分组成，分别是基于 EFK（Elasticsearch、Fluentd、Kibana）的容器日志搜集模块，以及基于 Kafka 的监控告警模块，这两个部分相互协作，为系统的稳定运行提供了坚实的保障。

（1）容器日志搜集模块

a) Fluentd

Fluentd 作为日志搜集的前端组件，负责从各个容器中收集日志数据。在容器化的环境中，每个容器都会产生大量的日志信息，这些日志记录了容器内部的运行状态、应用程序的执行情况等重要信息。Fluentd 具有高度的灵活性和可扩展性，项目通过集成 Kafka 插件和 Kubernetes 插件实现了 Kubernetes 运行容器的日志捕捉。

项目中通过预设定初始化脚本，系统将自动以 Daemonset 的形式在每一个 Node 节点启动 Fluentd 组件。Fluentd 一旦收集到日志数据，Fluentd 会对其进行格式化和预处理，将不同格式的日志统一转换为标准的 JSON 格式，并将处理

后的日志数据高效地传输到 Elasticsearch 中进行存储。

b) Elasticsearch

Elasticsearch 是一个分布式的搜索和分析引擎，它为日志数据提供了高效的存储和快速的检索能力。在接收到 Fluentd 传输过来的日志数据后，Elasticsearch 会将其存储在分布式的索引中。本项目中根据 Pod 的资源名称和日期组合动态生成了分布式索引，极大的提高了查找效率。

c) Kibana

Kibana 为用户提供了一个直观的界面，用于可视化和分析 Elasticsearch 中存储的日志数据。通过 Kibana 的仪表盘功能，用户可以创建各种类型的图表和报表，如柱状图、折线图、饼图等，直观地展示日志数据的统计信息和趋势。通过页面友好的 Kibana，用户可以更加方便快捷的查询系统日志，从而定位到具体的异常场景。

(2) 监控告警功能模块

监控告警模块是基于 Kafka 实现的功能模块。在该模块中，由使用 Kafka 插件的 Fluentd 充当生产者，消费者开发在告警服务模块中。

在监控告警数据链路中，Fluentd 生产者将每日搜集 Kubernetes 中运行的 Pod 的日志，并在 Kubernetes 插件的基础上处理成含有 Pod 元数据的、格式化的 JSON 对象，然后将数据推入至 Kafka 中。

Kafka 将每日持久化 Pod 的运行日志，通过可移动的偏移量实现消息消费。

消费者开发在告警模块中。消费者从 Kafka 拉去消息，并解析消息中的具体内容，并过滤出日志等级大于 Info 的记录消息。对于这部分消息，告警模块会将日志以邮件的形式通过 Gomail 组件发送给容器的构建者，从而实现了完整的监控告警闭环。

3.1.5 基建层

在本项目的架构体系里，基建层堪称整个系统稳定运行的坚实底座，它引入了一系列必不可少的中间件。这些中间件各自承担着独特的功能，如同紧密协作的团队成员，共同为项目的顺畅运转保驾护航。

基建层所选用的中间件涵盖了构建镜像的 Docker、负责容器编排的 Kubernetes、消息队列 Kafka、对象存储阿里云 OSS、结构化数据库 MySQL、高速非结构化缓存 Redis 以及支持倒排索引的 Elasticsearch。Docker 让应用程

序及其依赖能够以镜像的形式被封装，是 CI/CD 的核心工具。Kubernetes 负责容器编排，通过 Kubectl 命令一键实现 CD 持续部署的流程。Kafka 作为消息队列，为不同组件之间的通信搭建了高效的桥梁，实现了异步通信，解耦了模块之间的依赖关系。阿里云 OSS 提供了安全、可靠且低成本的对象存储服务，满足了项目对海量非结构化数据存储的需求。MySQL 作为经典的结构化数据库，负责存储和管理项目中的各类业务数据，保证了数据的完整性和一致性。Redis 以其高速的读写性能，为系统提供了缓存支持，显著提升了数据的访问速度。Elasticsearch 凭借其强大的倒排索引功能，提供了高效的日志检索能力。

这些中间件的有机组合和合理运用，共同构建起了一个稳固而扎实的基建层，为上层应用层提供了一个稳定、可靠的运行环境。它屏蔽了底层基础设施的复杂性，使得上层应用层能够专注于业务逻辑的开发和实现，为整个项目的成功实施提供了坚实的保障。

3.1.6 第三方服务

本系统增设第三方服务层用于实现系统中的部分功能，使系统成为更加完善、设计合理的有机整体。

项目中使用阿里云 ESC 云服务器部署了开发环境所需的中间件：数据库 MySQL、缓存 Redis、Elasticsearch 等。通过云端部署中间件的形式，使中间件可以稳定的运行云服务器中，从而与项目解耦，降低系统维护的成本。

使用 GitHub 和 Git 工具实现代码仓库的云端共享和记录，本项目源码现已经全部开源在 GitHub 网站上，并以 Git commit 的形式记录了开发工程中的问题和进度。

使用阿里云镜像仓库，作为云端镜像的存放空间，从而标准化了 Docker 镜像地推送流程吗，降低了用户的维护成本。

借助上述的第三方服务，本系统构建了一个完整、流畅且规范的开发和测试环境。在本地开发过程中，开发人员可以专注于业务逻辑的实现，而无需担心中间件的部署和维护问题。同时，通过 GitHub 和 Git 的版本控制功能，本作者详细记录了开发过程中的思考和开发进度，为项目的后续迭代和优化提供了宝贵的经验。

3.2 数据库设计

3.2.1 数据库概念结构设计

本部分使用 E-R 图的方式描述数据库的概念模型。实体类型包括用户（user 表）、用户 OSS 账号（user_oss 表）、用户 Docker 账号（user_docker 表）、用户 GitHub 账号（user_github 表）、团队（team 表）、团队请求（team_request 表）、Dockerfile 文件（user_dockerfile 表）、Docker 镜像记录（user_docker_image）、Kubernetes 资源（user_k8s_resource 表）、Kubernetes 操作记录（user_k8s_resource_log 表）。

实体类型中每一个实体均包含 ID、Created_at、Updated_at、Deleted_at 属性，以分别标识实体的自增 ID、实体数据的创建时间、实体数据的更新时间、实体数据的软删除时间。

（1）用户实体

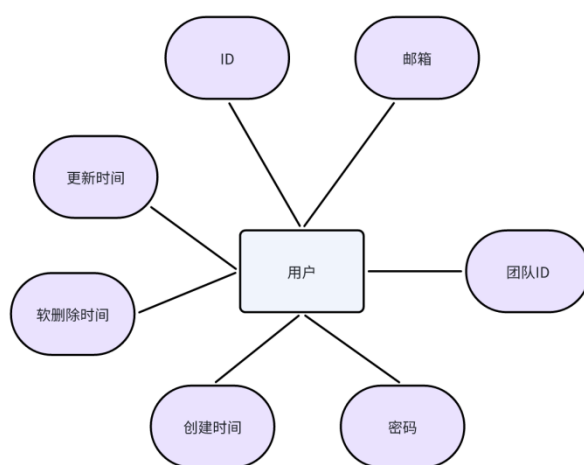


图 3.2 用户实体

如图 3.2 用户实体所示，用户实体包括 ID、创建时间、更新时间、软删除时间、邮箱、团队 ID、密码 7 个属性。

用户实体表示用户信息，其中密码属性以暗文加密形式存储。

(2) 用户 OSS 账号实体

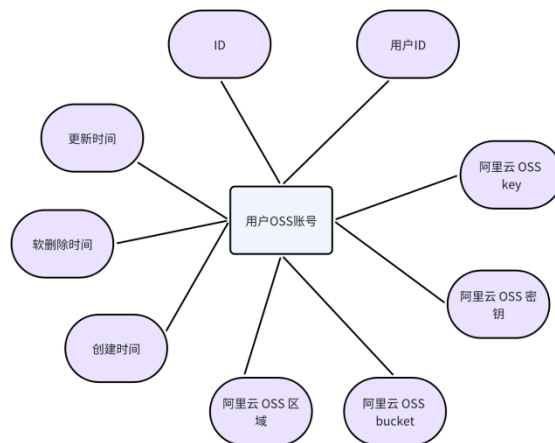


图 3.3 用户 OSS 实体

如图 3.3 用户 OSS 实体所示，用户 OSS 实体包括 ID、创建时间、更新时间、软删除时间、用户 ID、阿里云 OSS key、阿里云 OSS 密钥、阿里云 OSS bucket、阿里云 OSS 区域 9 个属性组成。

用户 OSS 实体表示用户绑定的 OSS 账号信息。用户实体与用户 OSS 实体呈 1 对 1 的绑定关系。其中阿里云 OSS key、阿里云 OSS 密钥、阿里云 OSS bucket、阿里云 OSS 区域 4 个属性均为必要属性，不允许为 NULL。

(3) 用户 Docker 账号实体

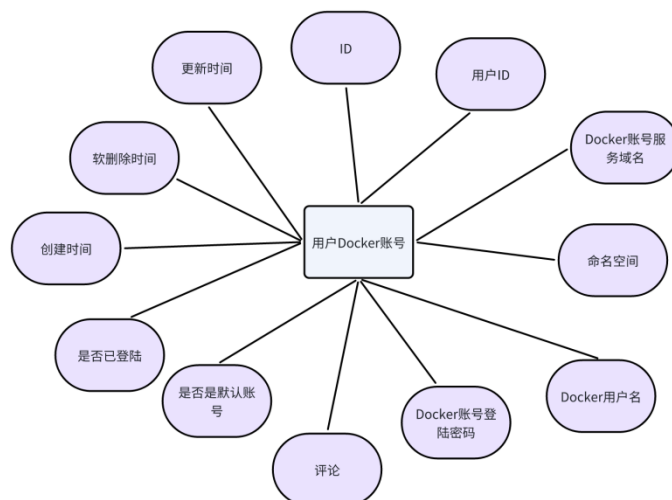


图 3.4 用户 Docker 账号实体

如图 3.4 用户 Docker 账号实体所示，用户 Docker 账号实体包括 ID、创建时间、更新时间、软删除时间、用户 ID、Docker 账号服务域名、命名空间、Docker 用户名、Docker 账号登录密码、评论、是否是默认账号、是否已登录一种 12 个属性组成。

用户 Docker 账号实体表示用户绑定的 Docker 账号信息。其中 Docker 账号服务域名、命名空间、Docker 用户名、Docker 账号登录密码是必须属性，不允许为 NULL。用户实体与用户 Docker 账号实体是 1 对 N 的关系。

(4) 用户 GitHub 账号实体

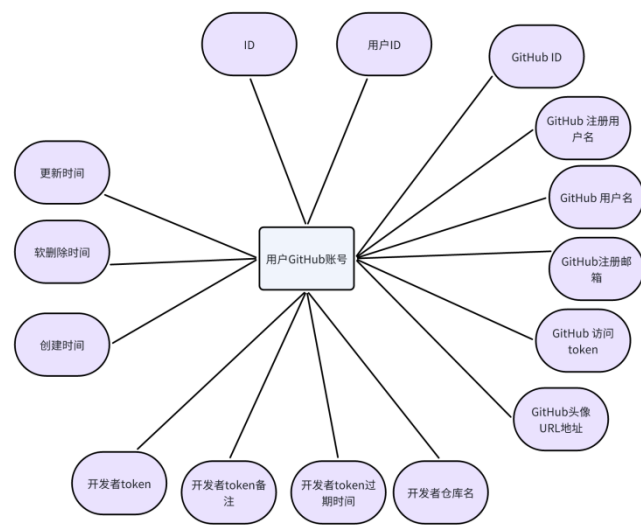


图 3.5 用户 GitHub 实体

如图 3.5 用户 GitHub 实体所示，用户 GitHub 实体包括 ID、创建时间、更新时间、软删除时间、用户 ID、GitHub ID、GitHub 注册用户名、GitHub 用户名、GitHub 注册邮箱、GitHub 访问 token、GitHub 头像 URL 地址、开发者仓库名、开发者仓库名 token 过期时间、开发者 token 备注、开发者 token 共计 15 个属性。

用户 GitHub 实体表示用户的 GitHub 账号信息，与用户实体是 1 对 1 的关系。其中 GitHub 开头的实体属性均通过绑定 GitHub 账号时自动获取。

(5) 团队实体

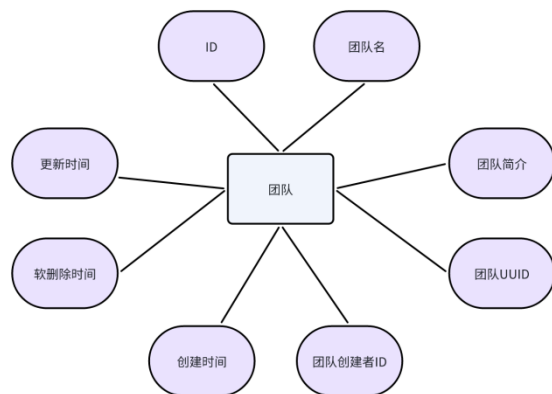


图 3.6 团队实体

如图 3.6 团队实体所示，团队实体包括 ID、创建时间、更新时间、软删除时间、团队创建者 ID、团队名、团队简介、团队 UUID，共计 8 个属性。

团队实体表示团队信息。用户实体和团队是 1 对 1 的关系。用户有且仅能加入一个团队中。

(6) 团队请求实体

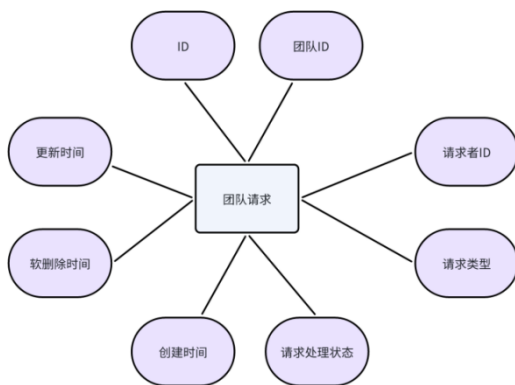


图 3.7 团队请求实体

如图 3.7 团队请求实体所示，团队请求实体包括 ID、创建时间、更新时间、软删除时间、团队 ID、请求者 ID、请求类型、请求处理状态共计 8 个属性。

团队请求实体表示用户在申请入队或者离队的申请请求记录。团队请求实体与团队实体的关系是 N 对 1 的关系，团队请求实体与用户实体是 N 对 1 的关系。用户可以发起多次申请。

(7) Dockerfile 文件实体

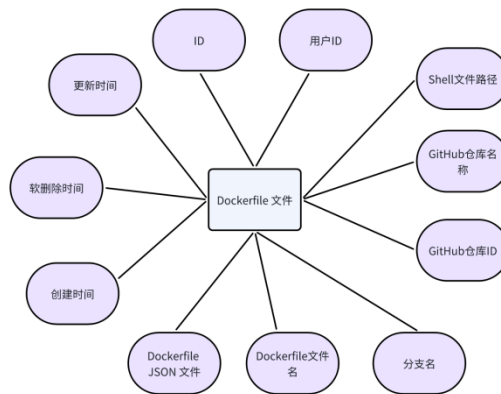


图 3.8 Dockerfile 文件实体

如图 3.8 Dockerfile 文件实体所示，Dockerfile 文件实体包括 ID、创建时间、更新时间、软删除时间、用户 ID、Shell 文件路径、GitHub 仓库名称、GitHub 仓库 ID、分支名、Dockerfile 文件名、Dockerfile JSON 文件共计 11 个属性。

Dockefile 文件实体表示 Dockerfile 文件和代码仓库的绑定关系，记录的是一个 GitHub 代码仓库下的一个具体的分支中的用户自定义 Dockerfile 文件。Dockerfile 文件实体与代码仓库 ID 的具体分支的关系是 1 对 1。

(8) Docker 镜像记录实体

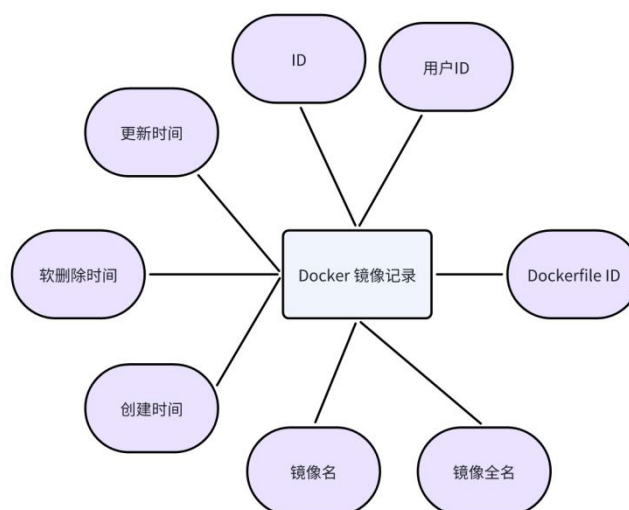


图 3.9 Docker 镜像记录实体

如图 3.9 Docker 镜像记录实体所示，Docker 镜像记录实体包括 ID、创建时间、更新时间、软删除时间、用户 ID、Dockerfile ID、镜像全名、镜像名，共计 8 个属性。

Dockerfile 镜像记录实体表示 Docker 镜像构建完成后记录数据，与 Dockerfile 实体的关系是 N 对 1 的关系。

(9) Kubernetes 资源实体

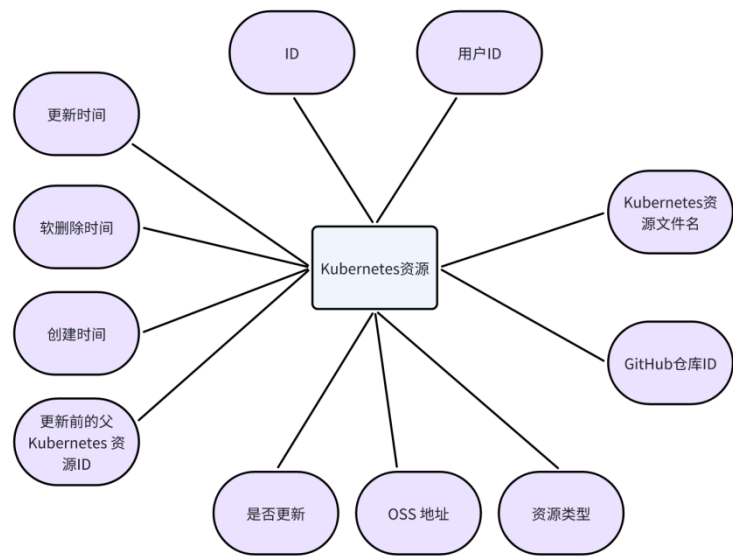


图 3.10 Kubernetes 资源实体

如图 3.10 Kubernetes 资源实体所示，Kubernetes 资源实体包括 ID、创建时间、更新时间、软删除时间、用户 ID、Kubernetes 资源文件名、GitHub 仓库 ID、资源类型、OSS 地址、是否更新、更行前的 Kubernetes 资源 ID，共计 11 个属性。

Kubernetes 资源实体表示 Kubernetes 资源部署所需要的配置文件，与 GitHub 代码仓库的关系是 N 对 1。一个 GitHub 代码仓库可以有多个 Kubernetes 资源实体。

(10) Kubernetes 操作记录实体

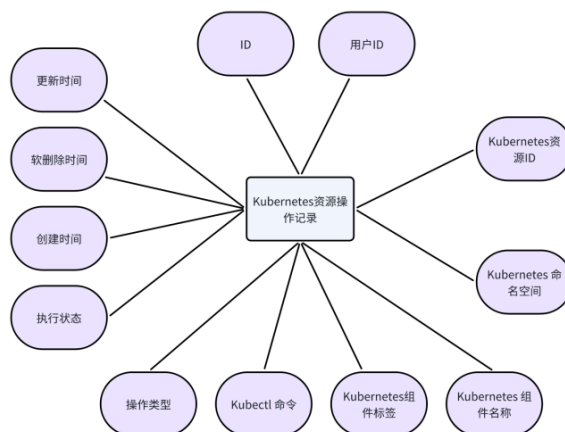


图 3. 11 Kubernetes 操作记录实体

如图 3. 11 Kubernetes 操作记录实体所示，Kubernetes 操作记录实体包括 ID、创建时间、更新时间、软删除时间、用户 ID、Kubernetes 资源 ID、Kubernetes 命名空间、Kubernetes 组件名称、Kubernetes 组件标签、KubectI 命令、操作类型、执行状态，共计 13 个属性。

Kubernetes 操作记录实体表示 Kubernetes 资源的具体操作日志，包括记录部署、删除、健康检查三中操作类型的 Pod 执行结果。Kubernetes 操作记录实体统计运行 Pod 的关键信息，由 Kubernetes 命名空间、Kubernetes 组件名称、Kubernetes 组件标签、KubectI 命令这几个属性表示。

Kubernetes 操作记录实体与 Kubernetes 资源实体的关系是 N 对 1。

3.2.2 数据库逻辑结构设计

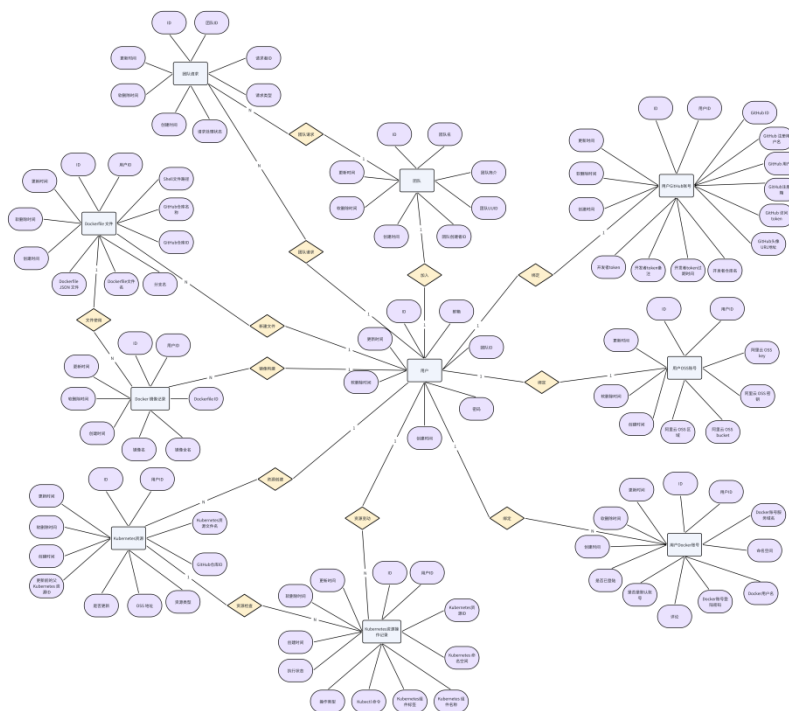


图 3.12 实体逻辑结构图

如图 3.12 实体逻辑结构图所示，用户实体、用户 OSS 账号实体、用户 Docker 账号实体、用户 GitHub 账号实体、团队实体、团队请求实体、Dockerfile 文件实体、Docker 镜像记录实体、Kubernetes 资源实体、Kubernetes 操作记录实体共计 10 个实体整体呈中心放射网状关联。其中位于关联图中心的用户实体，与除 Kubernetes 资源实体之外的 8 个实体都有直接关联关系。

在用户 GitHub 账号实体、用户 OSS 账号实体两个实体，其与用户实体正弦 1 对 1 的绑定关系，即表示一个用户只能绑定一个 OSS 账号和一个 GitHub 账号。对于用户 Docker 账号实体则于用户实体呈 N 对 1 的绑定关系，系统允许一个用户绑定多个 Docker 账号实体，且在不同的 Docker 账号间自由切换，通过“是否已登陆”、“是否是默认账号”两个熟悉进行控制。

对于团队实体和团队请求实体，二者是 1 对 N 的绑定关系，一个团队可以收到多个团队请求。同时团队实体与用户实体呈 1 对 1 的绑定关系，用户实体与团队请求实体呈现 1 对 N 的关联关系。一个用户有且仅能加入一个团队中，但是一个用户可以对不同的团队发起多次团队请求包括入队和出队两种状态。

对于 Dockerfile 文件实体和 Docker 镜像记录实体，二者呈现 1 对 N 的绑定

关系。一个 Dockerfile 文件实体可以被用于多次构建不同的 Docker 镜像记录实体。但 Dockerfile 文件实体的创建和 Docker 镜像的构建都由用户手动操作，因此与用户实体的关联关系呈现 N 对 1 的关联关系。

对于 Kubernetes 的资源文件实体和 Kubernetes 资源操作记录实体呈 1 对 N 的关联关系，一个 Kubernetes 资源文件可以被多次部署、删除、健康检查并被记录保存为 Kubernetes 资源操作记录实体。同时一个用户可以多次创建和操作 Kubernetes 的资源文件实体和 Kubernetes 资源操作记录实体，因此用户实体与二者呈现 1 对 N 的关联关系。

第4章 功能需求分析

4.1 登录注册功能

登陆注册模块系统使用的入口，只有用户完成了合法注册和登录才能进入系统正常使用系统功能。

对于注册功能需要经过两个过程才能实现完整的用户注册功能。在第一个过程用户需要输入合法的电子邮箱地址和用户密码，系统会将用户信息数据存入 Redis 缓存，然后向用户邮箱发送验证码。在第二个过程中，用户需要输入邮件中的验证码以向系统证明用户邮件的有效性。等待系统校验通过后便将 Redis 中的用户缓存写入 MySQL 进行持久化存储。

对于用户登录，用户需要输入注册时使用的电子邮箱和密码。在应用层登录注册模块校验通过之后，系统会向表示层返回一个 24 小时有效的 token 字符串。该 token 是用户 24 小时内的唯一表示，用于网关层的合法校验。后续在系统中的所有网络请求都需要在请求头携带 token 信息以通过网关层的合法用户校验。

如图 4.1 用户登录时序图所示是用户登录的完整流程，如图 4.2 用户注册时序图所示是用户注册的完整流程。

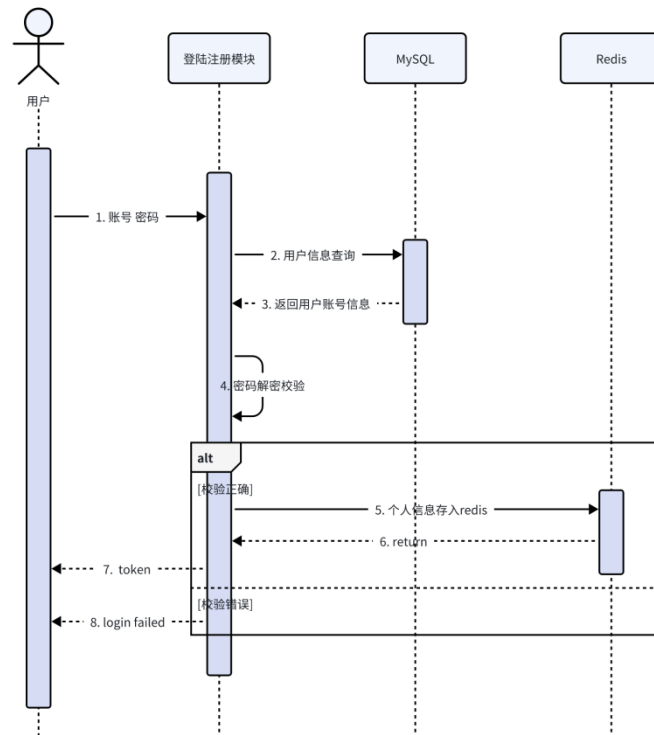


图 4.1 用户登录时序图

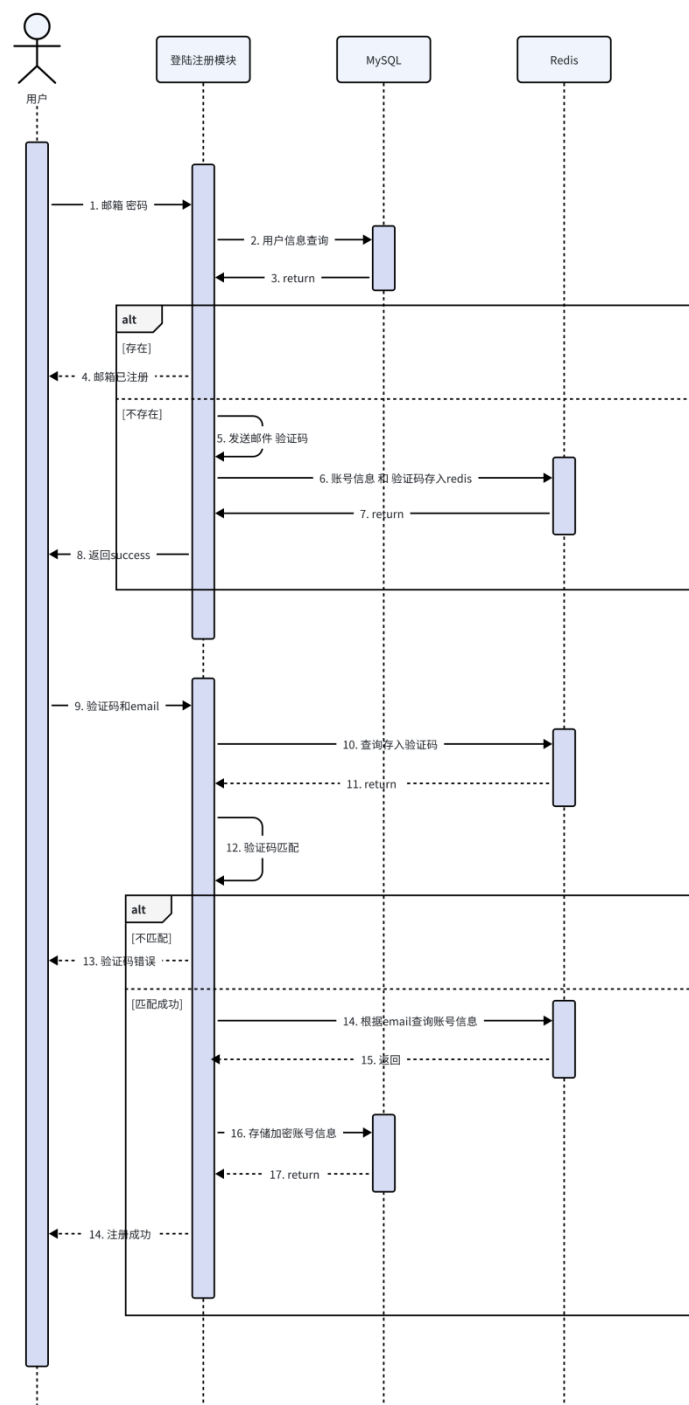


图 4.2 用户注册时序图

4.2 Auth 鉴权模块功能

Auth 鉴权是网关层的主要功能，用于拦截非法 HTTP 和 WebSocket 请求，并解析正常请求中的 token 信息，进而获取用户信息加载入请求上下文。表示层将

token 传入网关层，网关层根据 token 从 Redis 获取用户信息，如果 Redis 用户信息不存在则提示用户“登录过期，请重新登录”。Auth 鉴权流程如图 4.3 Auth 鉴权流程图所示。

网关层是应用层的拦截器，为应用层提供了标准且稳定的网络环境。

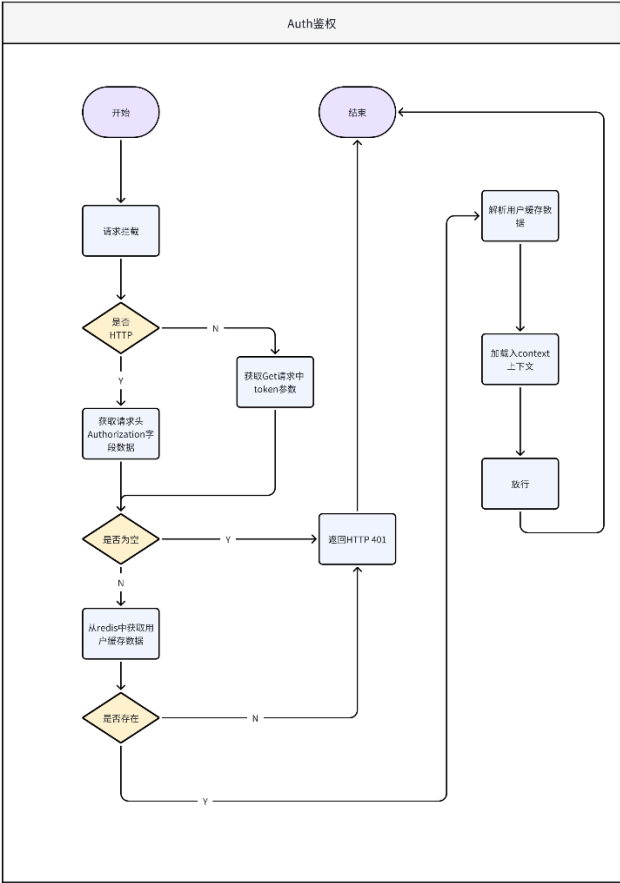


图 4.3 Auth 鉴权流程图

4.3 个人信息管理功能

在“个人信息”页面中，用户需要管理 OSS 账号、Docker 镜像仓库账号、GitHub 账号。通过用例图的形式可以直观反映用户需要操作的个人信息，个人信息管理用例图如图 4.4 个人信息用例图所示。

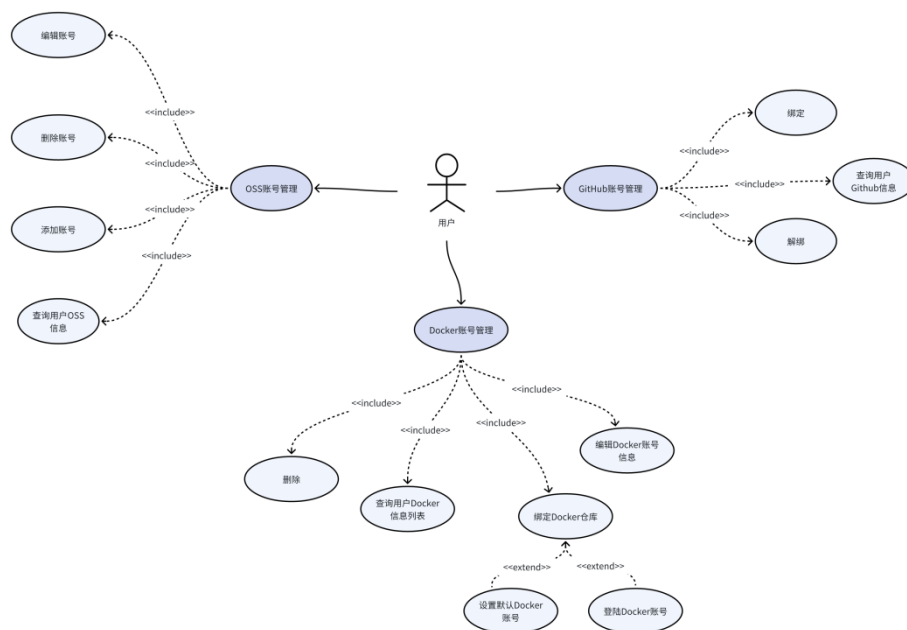


图 4.4 个人信息用例图

4.4 GitHub 账号绑定功能

GitHub 账号绑定是系统正常使用的前置流程，用户必须完成 GitHub 账号绑定才能使用系统的其他功能。“代码仓库管理”、“镜像构建”、“Kubernetes 部署”三个功能都依赖用户的 GitHub 账号才能正常使用。

在 GitHub 账号绑定中，需要由表示层向 GitHub 服务发起绑定请求，GitHub 账号将重定向到指定路由进行鉴权的回调校验。在回调路由接口中，接口会收到 GitHub 返回的 code。接口需要依据这个 code 向 GitHub 发起回调校验，从而获取用户的 GitHub Access token，进而获得用户的 GitHub 信息。

如图 4.5 GitHub 绑定时序图所示是 GitHub 绑定的完整时序流程。

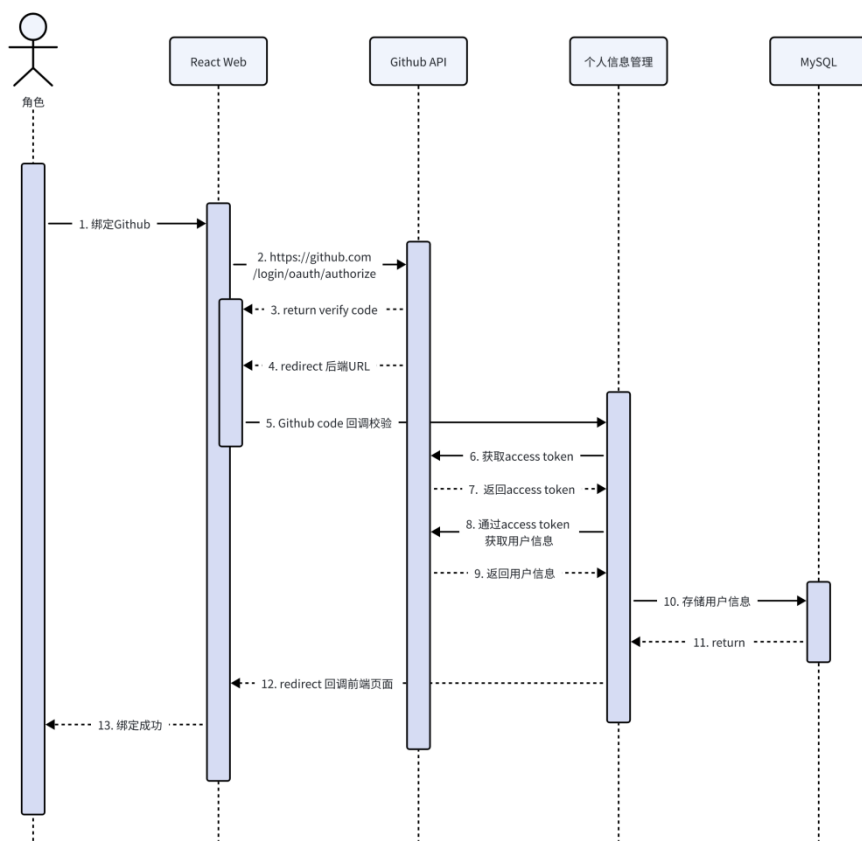


图 4.5 GitHub 绑定时序图

4.5 团队管理功能

团队是系统在业务层增设的一个概念。团队管理模块提供一系列的团队管理功能，为团队管理员和普通用户提供团队服务，包括团队查询、团队入队/离队申请，管理员审批，团队信息展示功能。

团队管理涉及功能用例如图 4.6 团队信息管理用例所示。

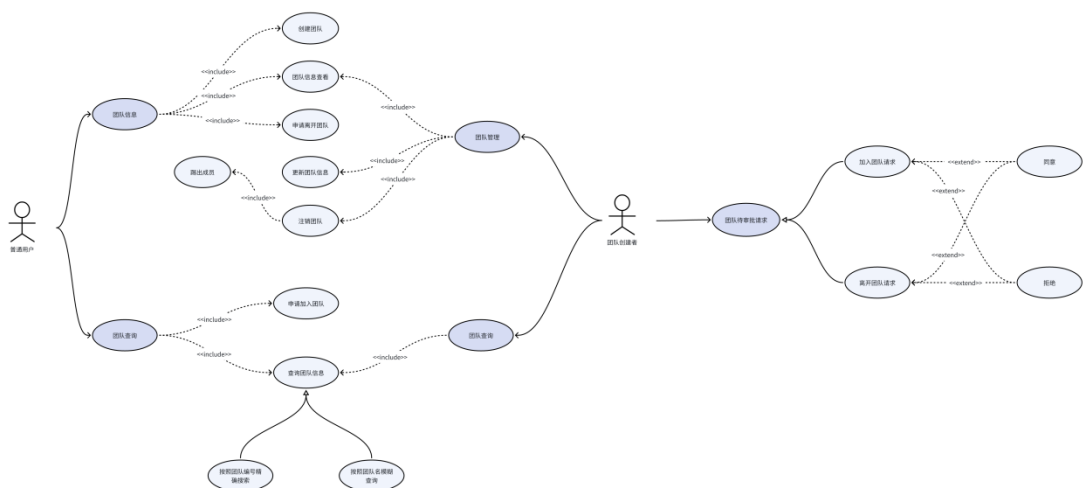


图 4.6 团队信息管理用例

4.6 团队申请功能

团队申请和团队管理中的核心功能。对于未加入团队的普通用户，其可以发起入队请求。该入队请求将以邮件的形式发送给团队管理员，由团队管理员审核通过之后向用户发送回调邮件，以告知用户审评结果。对于已入队的用户，提供离队申请功能。团队用户可以发起离队申请，同样在经过团队管理员审核通过之后受到离队审批结果邮件。团队申请流程如图 4.7 团队申请流程图所示。

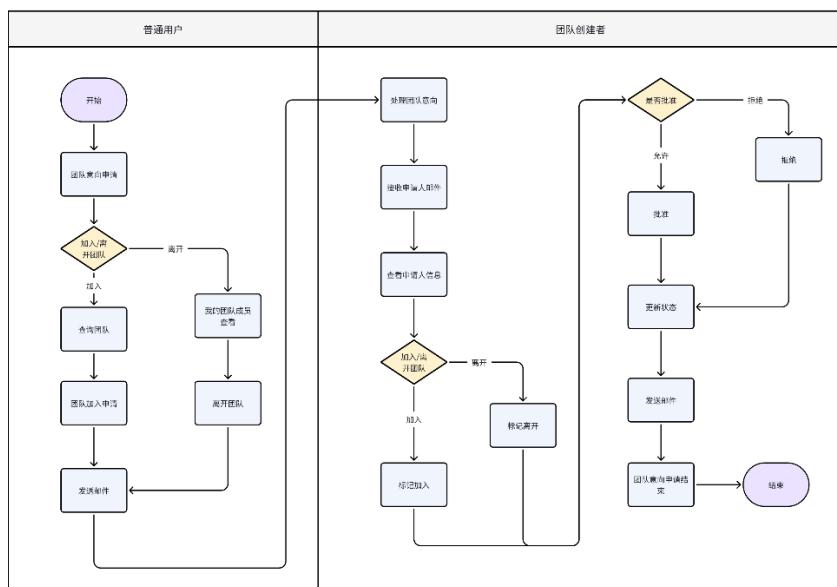


图 4.7 团队申请流程图

4.7 代码仓库渲染功能

在“代码仓库管理”页面，表示层通过 GitHub Open API 获取用户的 GitHub 仓库信息，GitHub 仓库中的文件结构按照标准文件目录渲染出来，并对具体文件进行特定的格式化渲染和 Git 提交信息渲染。

在调用 GitHub Open API 过程中，表示层需要向应用层获取用户绑定的 GitHub 账号 Access token 从而才能获取用户的 GitHub 代码仓库信息。表示层、应用层和 GitHub 服务的时序调用图如图 4.8 代码仓页面渲染时序图所示。

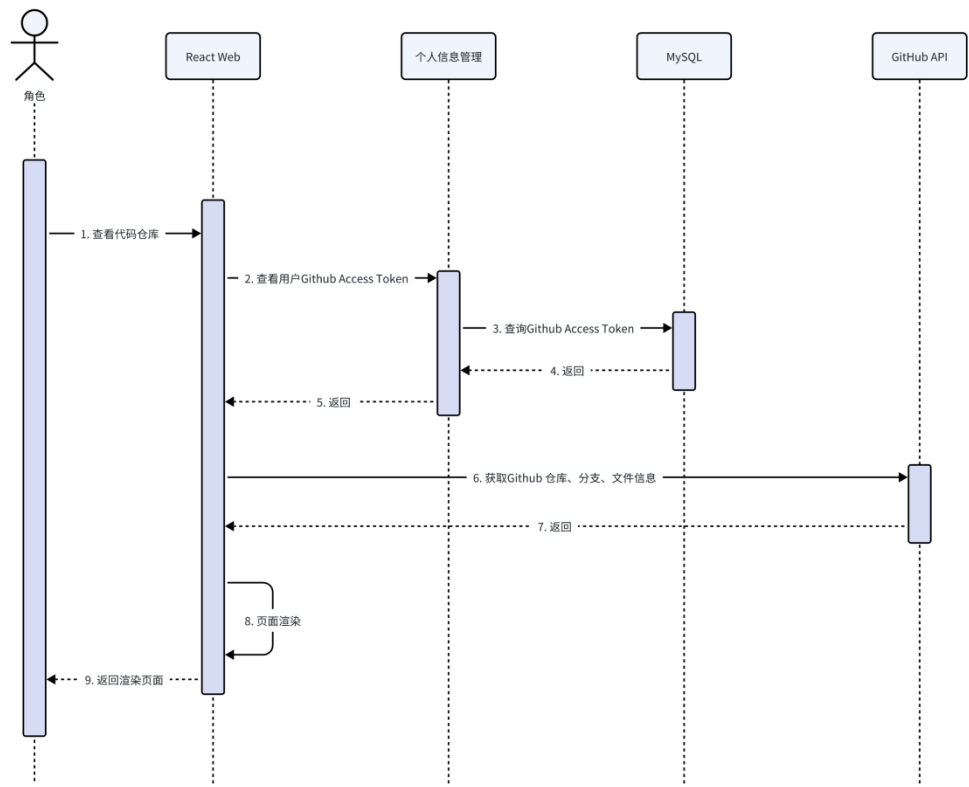


图 4.8 代码仓页面渲染时序图

4.8 Docker 镜像流水线构建功能

Docker 镜像的持续集成 CI 和持续交付 CD 是“镜像构建”页面中的核心功能。Docker 镜像构建分由 5 个部分组成，分别是“检查 Docker 账号”、“克隆代码仓库”、“生成 Dockerfile 文件”、“构建镜像”、“构建完成” 5 个部分。

镜像构建的 5 个部分由表示层控制构建进度，由应用层负责镜像 CI/CD 的具体逻辑，具体流程如图 4.9 镜像构建流程时序图所示。

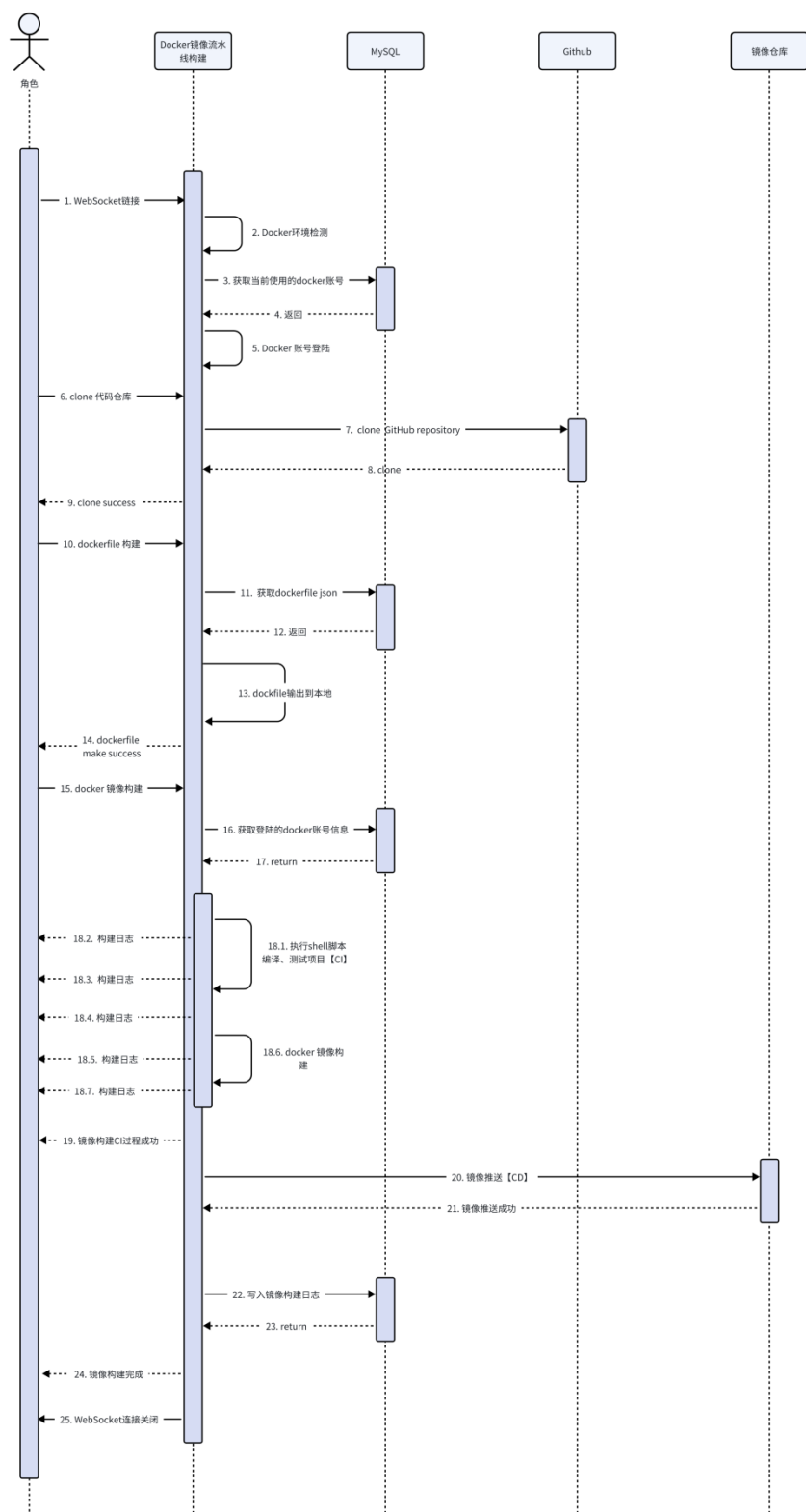


图 4.9 镜像构建流程时序图

4.9 Kubernetes 资源信息管理功能

在“Kubernetes 部署”页面中，其功能主要由三个部分组成，如图 4.10 Kubernetes 部署用例图所示分别是：“Kubernetes 资源管理”、“Docker 镜像版本查看”、“Kubernetes 控制面板”三个部分。

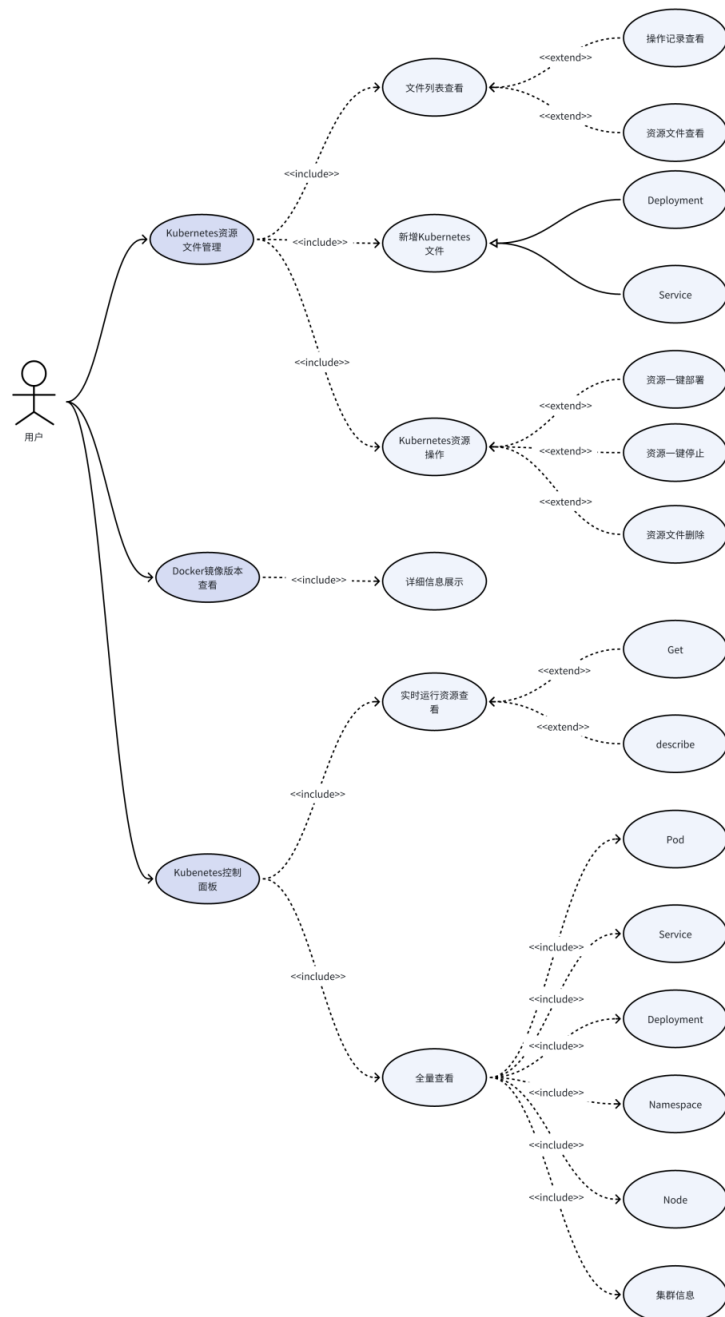


图 4.10 Kubernetes 部署用例图

4.10 Kubernetes 资源配置文件保存和更新功能

其中 Kubernetes 资源配置文件的保存和更新是 Kubernetes 资源信息管理的核心。文件保存和更新均通过 OSS 实现文件的云端存储。其中文件更新在应用层开发了链式更新功能。应用层将追踪被更新文件和更新后文件的父子关系，并将关系变化的转变写入 MySQL 中，保证文件更新的可回溯性和可监控性。

Kubernetes 资源配置文件的保存和更新如图 4.11 Kubernetes 资源配置文件保存时序图、图 4.12 Kubernetes 资源配置文件更新时序图所示。

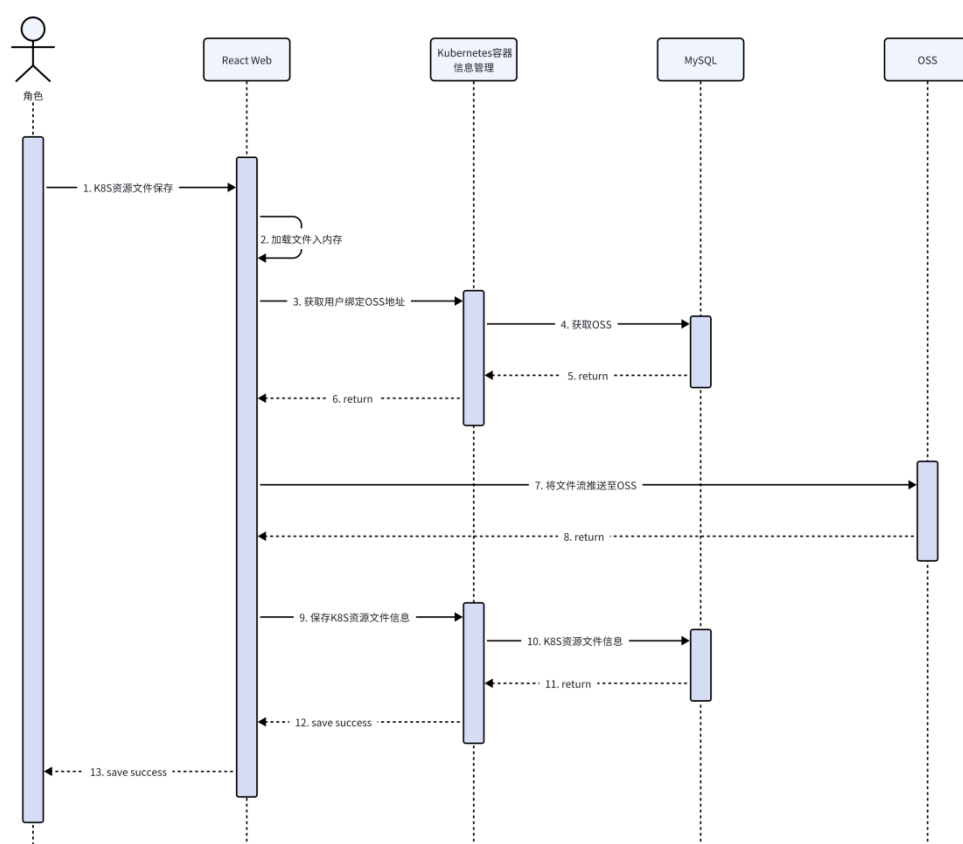


图 4.11 Kubernetes 资源配置文件保存时序图

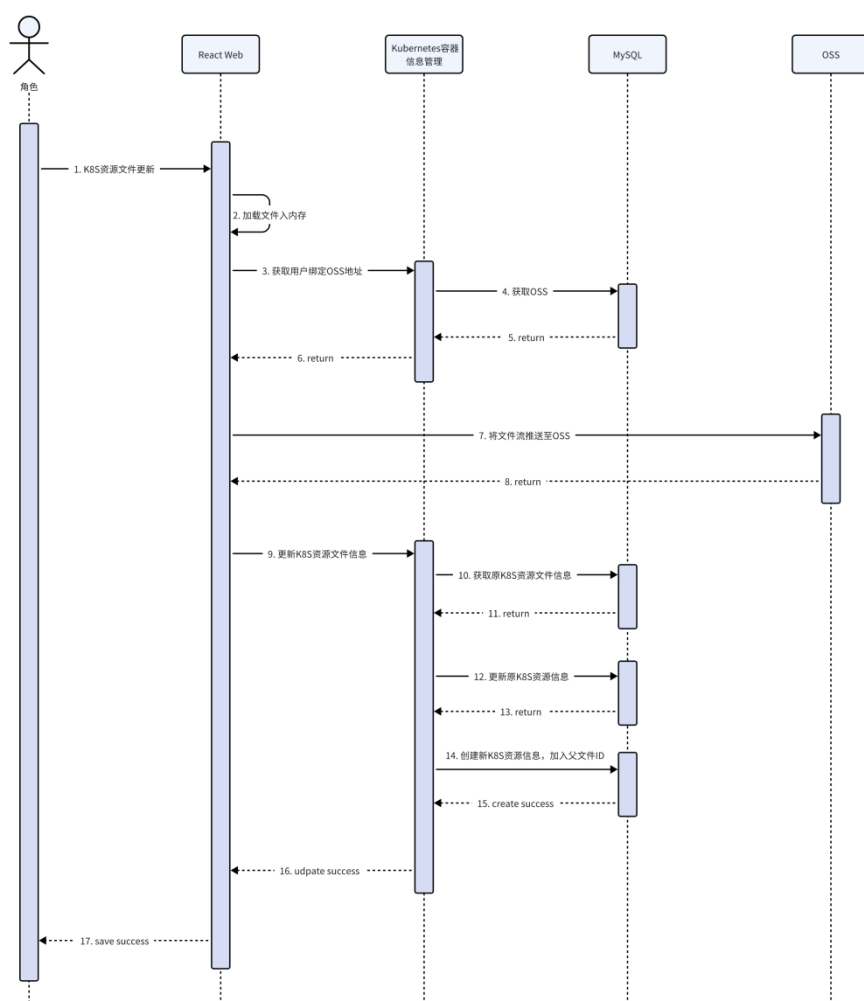


图 4. 12 Kubernetes 资源配置文件更新时序图

4.11 Kubernetes 资源配置文件渲染功能

对于 Kubernetes 资源配置文件的渲染，表示层会从 OSS 拉去配置文件，并加载入本地内存中，从而实现 0 数据写入磁盘的文件渲染能力。同时表示层将配置文件以标转 YAML 格式文件进行渲染，以方便用户直观查看配置文件的详细信息。渲染流程如图 4. 13 Kubernetes 资源配置文件渲染时序图所示。

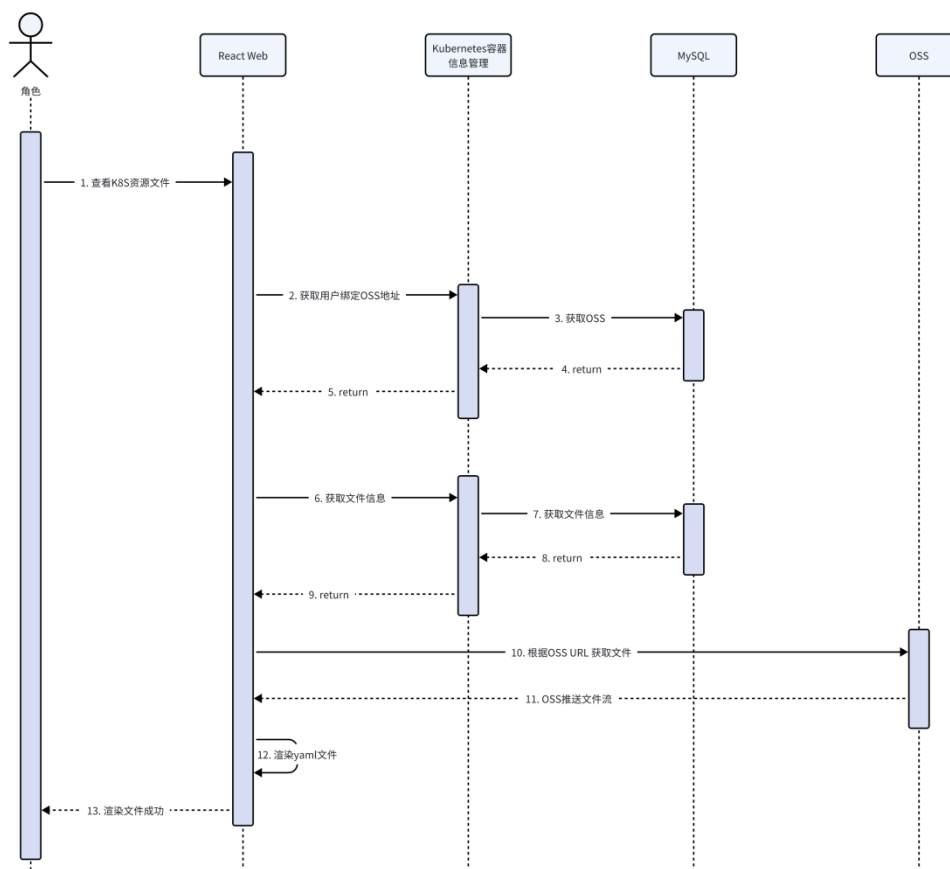


图 4. 13 Kubernetes 资源配置文件渲染时序图

4.12 Kubernetes 资源操作功能

Kubernetes 资源操作主要由“Kubernetes 控制面板”实现其中的功能。Kubernetes 资源操作实现的功能包括：“Docker 镜像的持续部署 CD”、“容器删除”、“查看所有 Pod”、“查看所有 Service”、“查看所有 Deployment”、“查看所有 Namespace”、“查看集群信息”、“查看节点状态”、“Get 查看具体资源”、“Describe 查看具体资源”。

在“Kubernetes 控制面板”中，用户可以在表示层页面选择具体的操作按钮，然后通过 WebSocket 传递消息给 Kubernetes 容器编排模块，然后等待 Kubernetes 执行结果的返回与渲染。操作流程如图 4. 14 Kubernetes 资源操作流程图所示。Kubernetes 容器编排模块会解析 WebSocket 消息，然后自行判断执行何种 Kubectl 命令进而实现 Kubernetes 资源的具体操作。

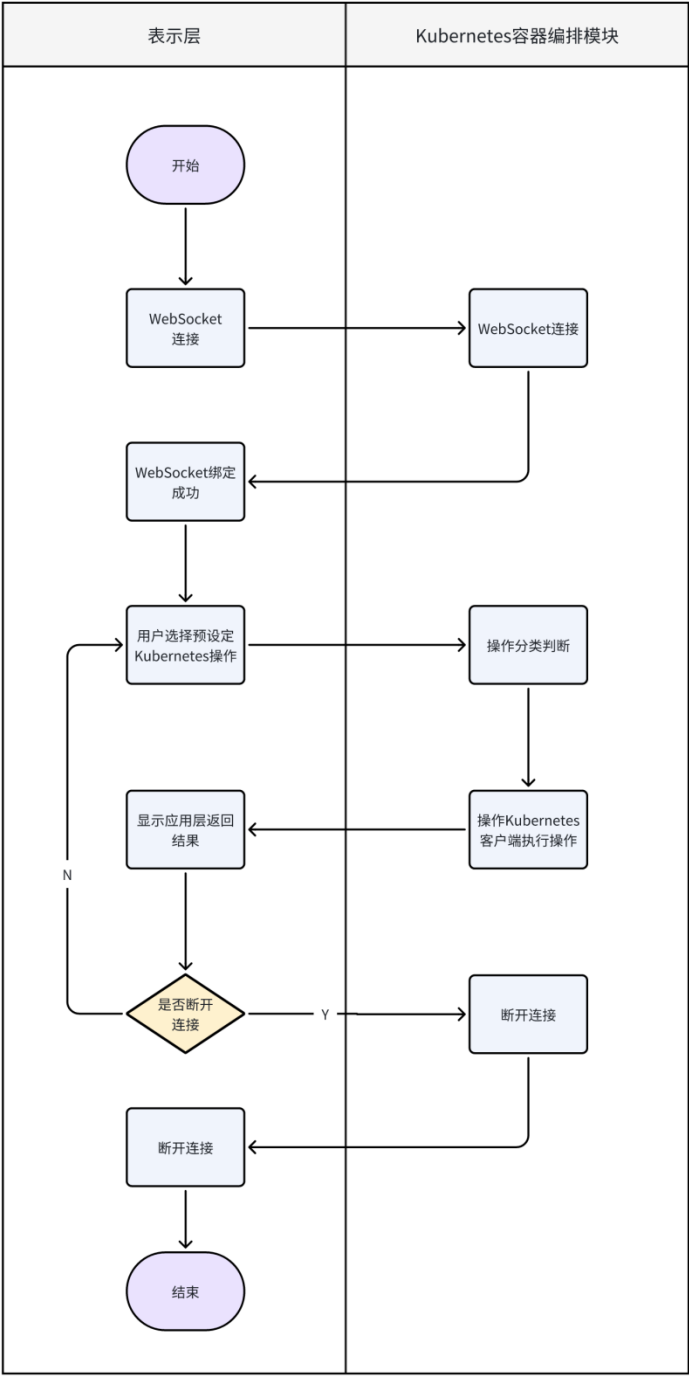


图 4.14 Kubernetes 资源操作流程图

4.13 Kubernetes 资源定时监控功能

对于团队内已经运行 Kubernetes 资源，系统将启动一个定时任务，在周期时间内对 Kubernetes 运行资源进行健康检测。对于正在运行的 Kubernetes 资源，应

用层将向表示层主动推送 WebSocket 消息，以让表示层渲染出实时运行的 Kubernetes 资源，进而便于系统用户观察 Kubernetes 资源状态。在定时任务中，Kubernetes 资源的状态信息将被同步写入 MySQL 数据库中，用户可以通过表示层的检索模块查看 Kubernetes 的历史运行状态记录，进而发现 Kubernetes 资源的潜在问题。Kubernetes 资源定时监控时序图如图 4.15 Kubernetes 资源定时监控时序图所示。

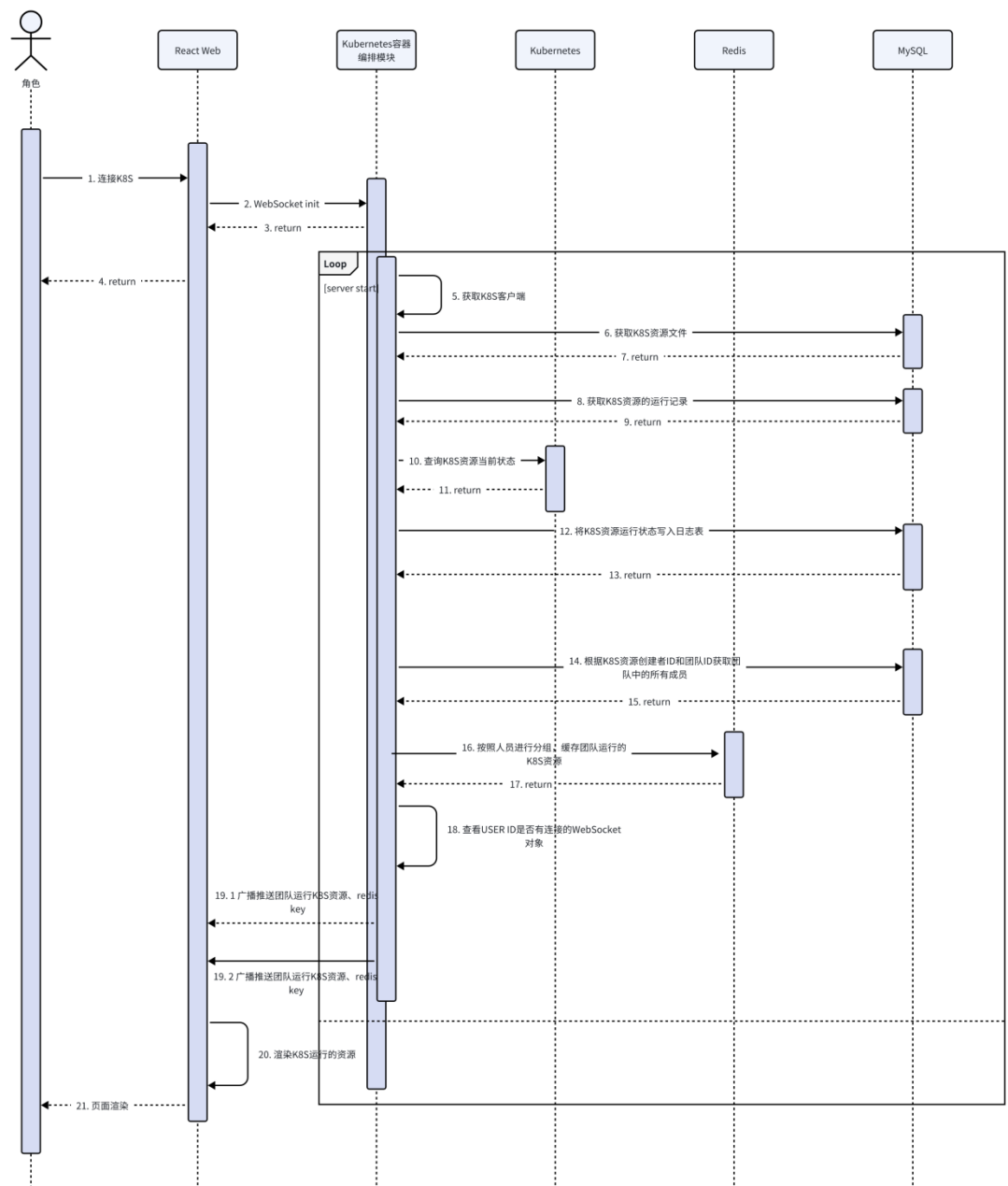


图 4.15 Kubernetes 资源定时监控时序图

4.14 EFK 日志搜集功能

本系统使用 EFK 技术实现 Kubernetes 容器的日志搜集和快速检索功能。系统将在每一个 Node 节点启动一个 Fluentd 容器用于容器搜集，然后由 Fluentd 将日志输入写入 Elasticsearch 中。Fluentd 将按照容器名和日期的组合形式动态创建 Index 从而实现容器日志的快速分类。最终由 Kibana 服务提供统一的日志快速检索服务, 便于用户使用 Elasticsearch 的倒排索引能力进行日志数据的快速定位。

EFK 日志搜集的数据流转图如图 4.16 EFK 数据流转图所示。

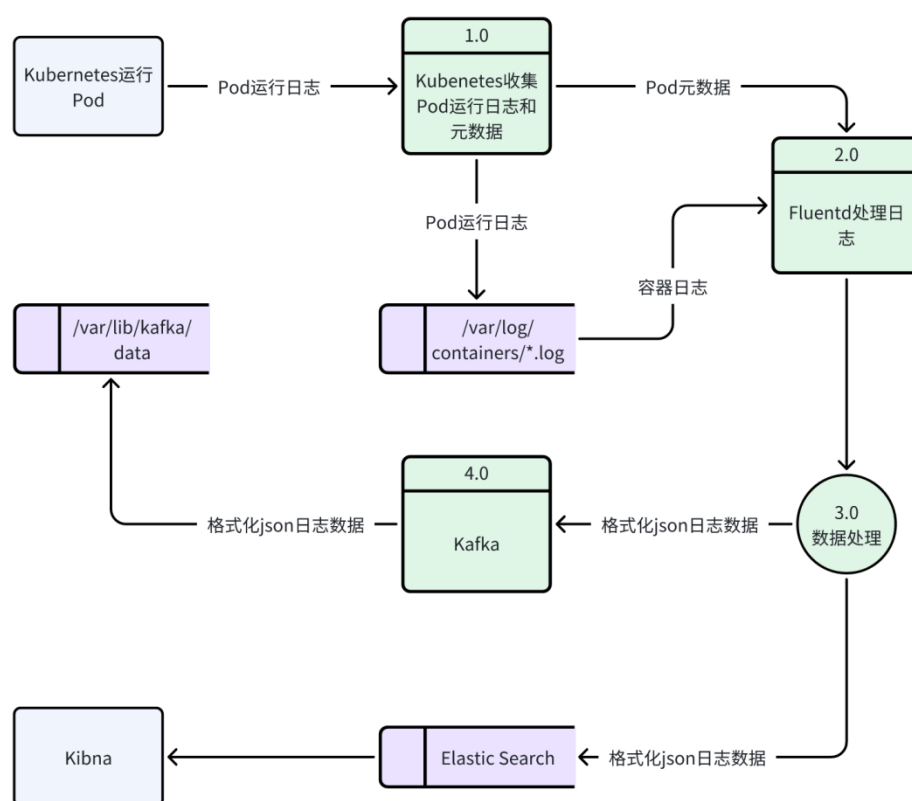


图 4.16 EFK 数据流转图

4.15 监控告警功能

如图 4.16 EFK 数据流转图所示，借助 Fluentd 和 Kafka 实现 Kubernetes 容器日志数据的异步发送，从而搭建基于 Kafka 的监控告警框架。Fluentd 作为生产者实时向 Kafka 推送容器的日志消息，本系统作为消费者实时从 Kafka 拉去消息进行异步消费。在消费消息的时候对日志消息的等级进行划分，对于日志等级

大于 Info 的日志消息进行邮件告警。

监控告警功能的整体流程如图 4.17 监控告警流程图所示。

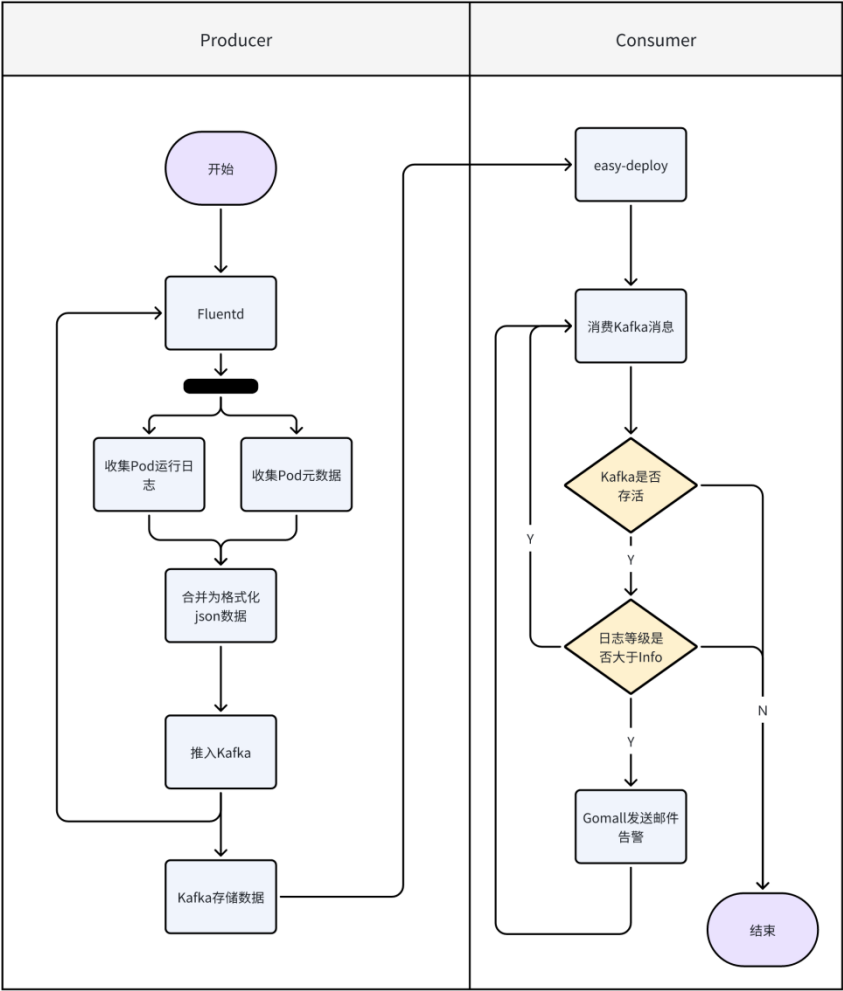


图 4.17 监控告警流程图

第5章 系统实现

5.1 系统开发环境

如第三章系统架构阐述所示，本系统按照表示层、网关层、应用层、监控层、基建层、第三方服务进行了 6 层划分。6 层架构之间相互关联但彼此解耦，使项目成为了一个完成有机的整体，如下表 4.1 开发环境是项目系统的开发环境。

表 4.1 开发环境

中间件	版本	用途
Go	v1.23	后端开发语言
Node	v18.20.5	前端运行环境
MySQL	v8.0.22	结构化存储
Redis	v6.2.14	非结构化缓存
Elasticsearch	v7.17.10	容器日志存储
Fluentd	v1.18	容器日志搜集
Kibana	v7.17.10	日志查询可视化前端
Kafka	v7.3.3	容器日志消息队列
Zookeeper	v7.3.3	Kafka 消息管理
Docker	v27.4.0	Docker 镜像构建
Kubernetes	v1.30.5	容器编排

5.2 系统功能实现

5.2.1 用户注册功能

用户进入系统首先进行登录注册页面，对于没有注册过的用户，用户可以点击注册按钮进行注册操作。注册页面如图 5.1 注册页面所示。



The registration form is titled "用户注册" (User Registration). It features a two-step progress indicator at the top: "1 填写信息" (1 Fill in information) in a blue circle and "2 验证邮箱" (2 Verify email) in a grey circle. Below the progress bar, there are two input fields: the first is for the email address, containing "111111111@qq.com", and the second is for the password, showing masked characters "....." with a lock icon on the left and an eye icon on the right. At the bottom of the form is a prominent blue button labeled "注册" (Register).

图 5.1 注册页面

用户需要输入邮箱和密码然后点击“注册”按钮进行注册操作。在点击注册按钮后系统会向用户发送一封含有验证码的邮件。用户需要手动输入验证码以完成最终的用户注册。

验证码邮件效果展示如图 5.2 验证码邮件所示。



图 5.2 验证码邮件

用户输入验证码效果如图 5.3 验证码校验所示。

用户注册

✓ 填写信息

2 验证邮箱

✉ 验证码

验证

图 5.3 验证码校验

5.2.2 用户登录功能

对于已经注册过的用户，用户可进入登录页面进行登录操作。用户需输入注册时使用的邮箱和密码才可以完成登录操作。用户登录成功后系统将自动跳转到系统首页。效果如图 5.4 登录页面所示。

用户登录

✉ 1111111@qq.com

🔒

👁

登录

图 5.4 登录页面

5.2.3 系统功能页面展示

系统从业务上划分为 5 个模块，分别是“个人中心”、“团队空间”、“代码仓库管理”、“镜像管理”、“Kubernetes 部署”。5 个模块以侧边栏的形式渲染在系统首页的左侧纵向展示列上，用户可点击具体的模块查看相应功能。用户点击左侧功能列表的 Tab 之后，具体功能页面将渲染在右侧窗口中。通过这种统用左侧功能列表栏、右侧核心功能展示页面的组合形式，降低用户使用本系统的学习成本，方便用户选择具体功能进行具体操作。系统首页效果图展示如图 5.5 系统首页所示。

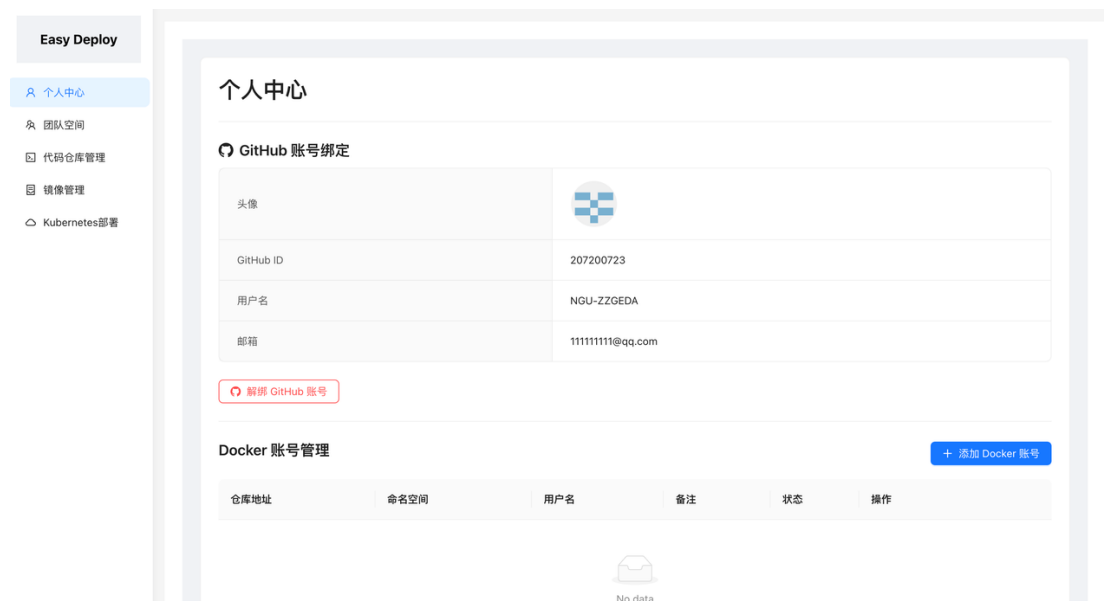


图 5.5 系统首页

用户进入系统首页默认进入“个人中心”页面。在“个人中心”模块中，用户可以绑定与解绑自己的 GitHub 账号，管理自己的 Docker 账户，已经绑定与解绑自己的 OSS 对象存储地址，效果展示图如图 5.6 个人中心所示。

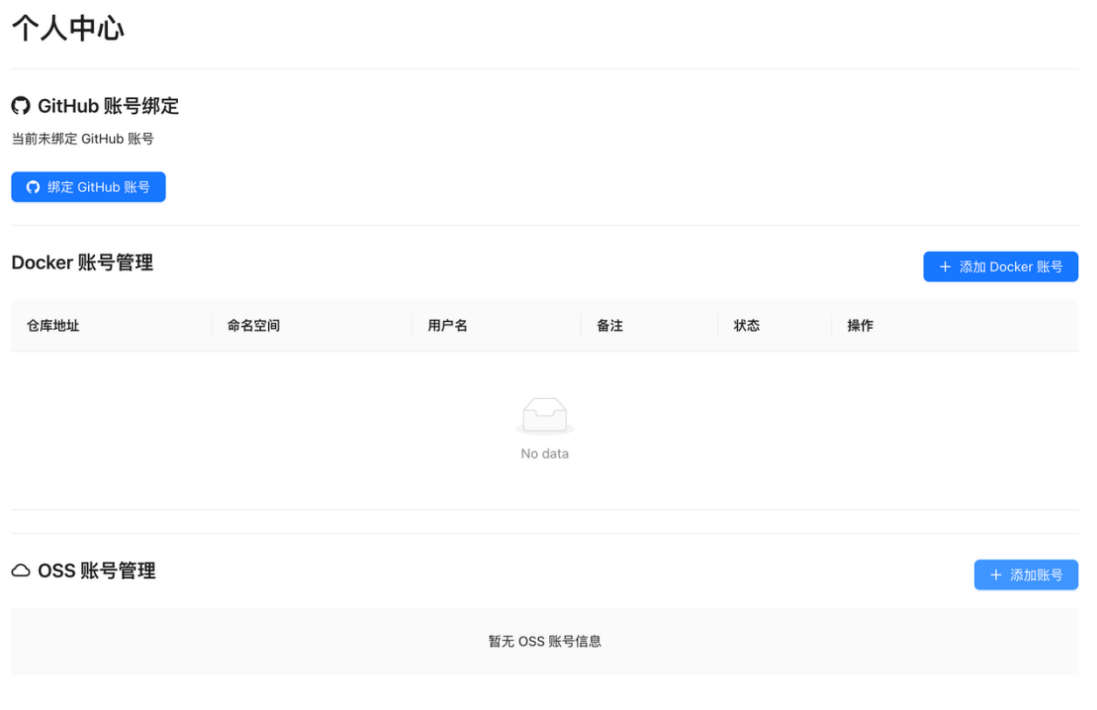


图 5.6 个人中心

用户可以点击“团队空间”Tab 然后进入团队空间页面。团队空间由三个部分组成，分别是“团队详情”、“团队查询”、“待审批请求管理”。用户可以根据自己的需求进行相应的操作。团队空间页面展示如图 5.7 团队空间所示。

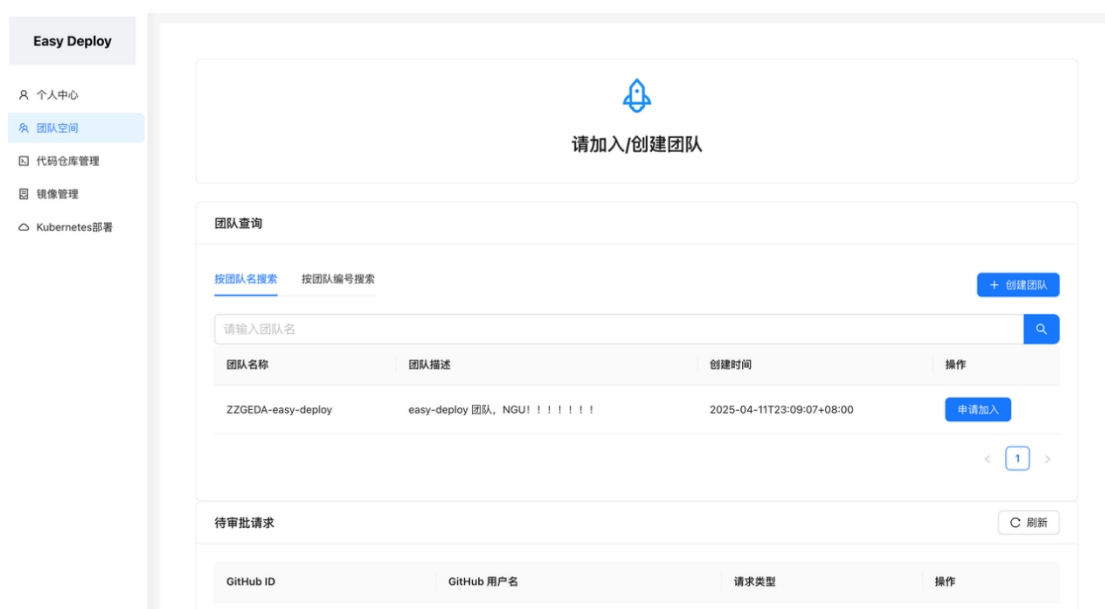


图 5.7 团队空间

用户可以点击“代码仓库管理”Tab 进入代码仓库管理页面。该页面展示用户绑定的 GitHub 账号中的代码仓库的详细情况。代码仓库管理页面分别展示用户创建和加入的所有代码仓库、具体代码仓库的具体分支的代码树状结构、代码文件的格式化渲染。代码仓库管理页面渲染效果如图 5.8 代码仓库管理所示。

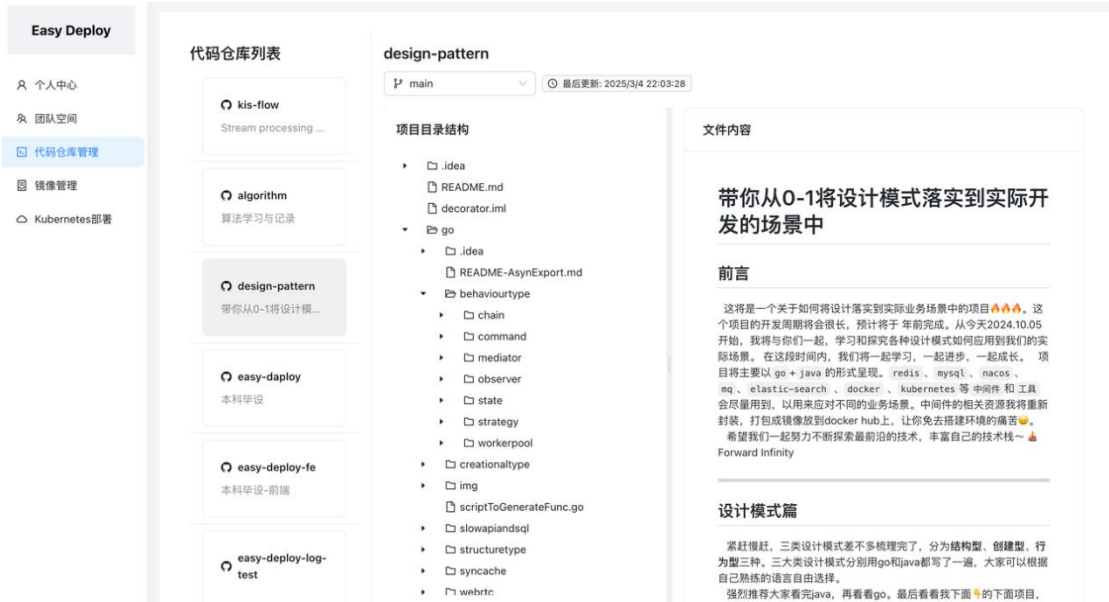


图 5.8 代码仓库管理

用户可以点击“镜像管理”Tab 进入镜像管理页面。镜像管理页面由两部分组成分别是代码仓库查询、镜像管理功能模块。用户可以在镜像管理页面实现 Docker 镜像的 CI/CD 过程。效果展示如图 5.9 镜像管理。

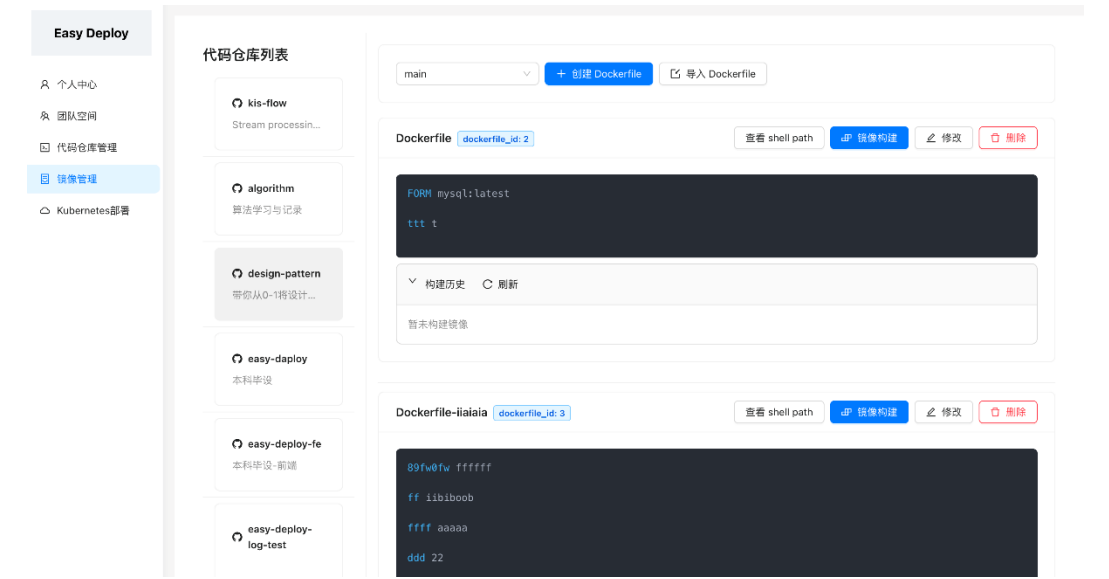


图 5.9 镜像管理

用户可以点击“Kubernetes 部署”Tab 进入 Kubernetes 部署页面。在该页面中用户在选择具体的代码仓库之后进行 Kubernetes 资源的部署任务,包括镜像查询、Kubernetes 资源文件管理、Kubernetes 资源管理等操作。页面展示如图 5.10 Kubernetes 部署所示。

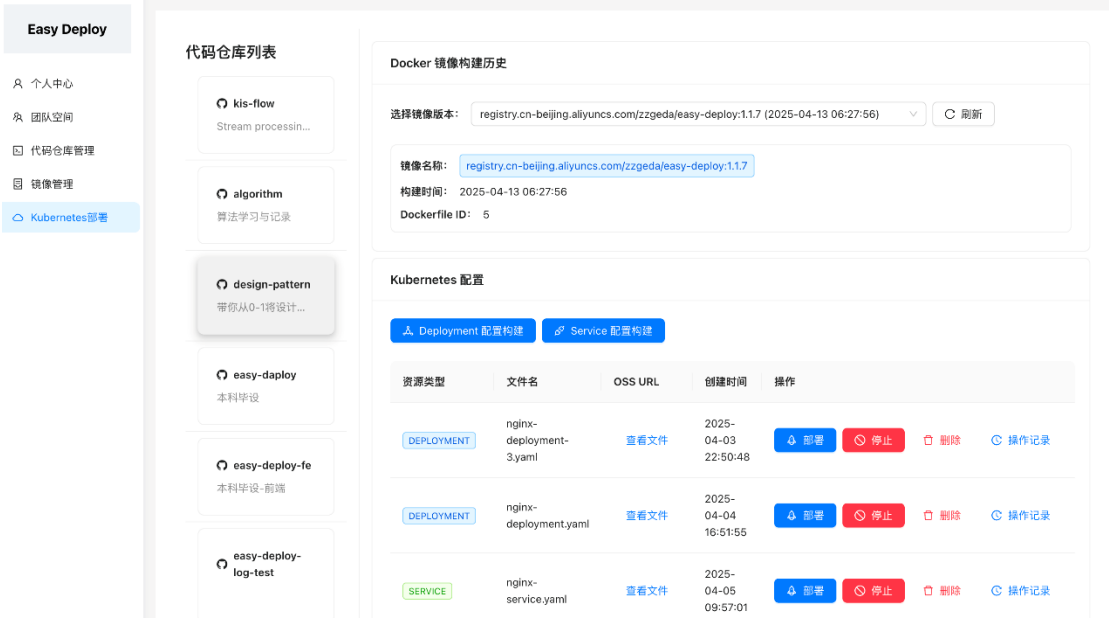


图 5.10 Kubernetes 部署

5.2.4 GitHub 账号管理

GitHub 账号管理在“个人信息”页面中。GitHub 账号绑定是用户正常使用本系统的前提。本系统与 GitHub 账号深绑定,“代码仓库管理”、“镜像管理”、“Kubernetes 部署”三个核心功能都需要获取用户的 GitHub 代码仓库才能正常使用。

在 GitHub 账号管理中,用户可以点击“绑定 GitHub 账号”进行 GitHub 账号绑定。用户绑定成功之后将在页面渲染用户的 GitHub 账号信息。如果用户想解绑本系统与自己 GitHub 账号的绑定关系,那么可以点击“解绑 GitHub 账号”进行解绑操作。

在 GitHub 账号绑定和解绑过程中,前后端均调用 GitHub API 以完成 GitHub 账号的授权和绑定操作。

GitHub 账号管理功能如图 5.11 GitHub 账户绑定、图 5.12 GitHub 账号解

绑所示。

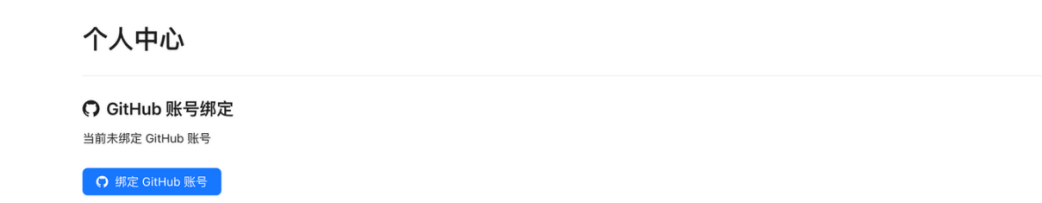


图 5.11 GitHub 账户绑定

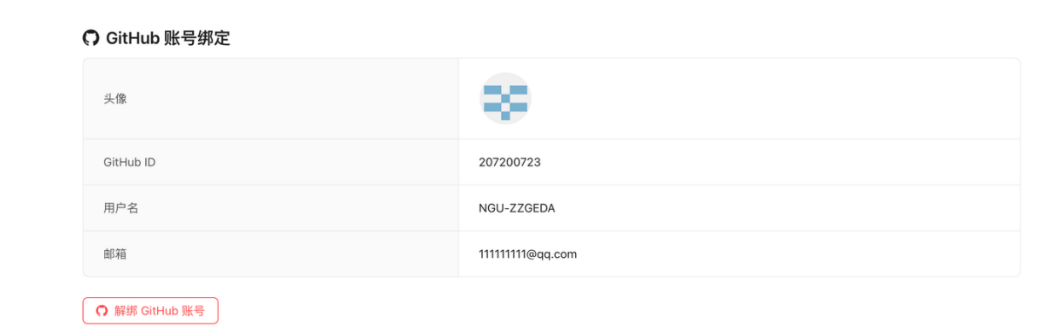


图 5.12 GitHub 账号解绑

5.2.5 Docker 账号管理

在 Docker 账号管理中，用户需要填写 Docker 账号相关的镜像仓库地址、命名空间、用户名、密码等信息。此信息用于镜像构建过程中的 CD 持续交付操作，将构建成功的镜像推入到远程镜像仓库中。

Docker 账号管理中，可以选择默认的 Docker 账号，并提供登录按钮一键实现 Docker 账号的本地登录。

页面效果渲染如图 5.13 Docker 账号查询、图 5.14 添加 Docker 账号所示。



图 5.13 Docker 账号查询

添加 Docker 账号

* 仓库地址

请输入 Docker 仓库地址

* 命名空间

请输入命名空间

* 用户名

请输入用户名

* 密码

请输入密码

* 备注

请输入备注信息

添加账号

取消

图 5.14 添加 Docker 账号

5.2.6 OSS 账号管理

在 OSS 账号管理中，用户需要填写 OSS Access Key ID 和 OSS Access Key Secret 两个必须信息。OSS 账号管理填写的信息将用于“Kubernetes 部署”中资源文件的上传操作。Kubernetes 资源上传将上传至 OSS，然后将文件 URL 路由存入后端数据库中，而非直接后端保存文件数据，从而价格数据流量成本。

OSS 账号管理中，用户有且仅能绑定一个 OSS 账号。如果用户需要更换 OSS 账号只能更新 OSS 账号信息。

效果展示图如图 5.15 OSS 账号信息展示、图 5.16 OSS 账号绑定所示。

OSS 账号管理

删除账号

编辑账号

Access Key ID	112211221122
Access Key Secret	112211221122
Bucket	auai
Region	oss-cn-beijing
创建时间	2025-04-16 15:34:46
更新时间	2025-04-16 15:34:46

图 5.15 OSS 账号信息展示

添加 OSS 账号

* Access Key ID

请输入 Access Key ID

* Access Key Secret

请输入 Access Key Secret

* Bucket

请输入 Bucket 名称

* Region

请输入 Region

添加账号 取消

图 5.16 OSS 账号绑定

5.2.7 团队创建

用户进入团队空间后可以自由选择加入团队或者创建自己的团队。对于创建团队，用户只需要填写团队名、团队简介即可，并支持在团队创建成功后修改。

团队创建成功后将自动重新渲染页面，将团队信息渲染在页面上，创建用户将自动成为团队管理员、允许修改团队信息、团队入队/离队审批。

效果展示如图 5.17 团队创建、图 5.18 团队简介-1 显示。

创建团队

* 团队名称

请输入团队名称

团队描述

请输入团队描述

创建团队

图 5.17 团队创建

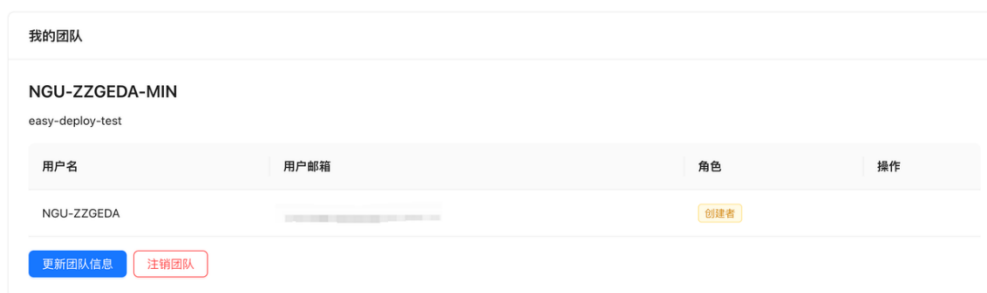


图 5.18 团队简介-1

5.2.8 团队查询和团队申请

对于想创建团队的用户，用户可以进入团队查询模块按照团队名进行模糊搜索，或根据团队唯一 UUID 进行团队编号的精确查询。

在查找到对应团队之后可以点击“申请加入”按钮想团队管理员发送入队申请。此时团队管理员会收到用户的“申请入队”邮件，并可在“待审批请求”模块处对申请用户进行审批操作。团队管理员可以允许或者拒绝用户的申请请求。

团队查询和申请入队邮件如图 5.19 团队查询、图 5.20 团队申请加入邮件所示。团队入队请求审批操作如图 5.21 团队入队审批所示。

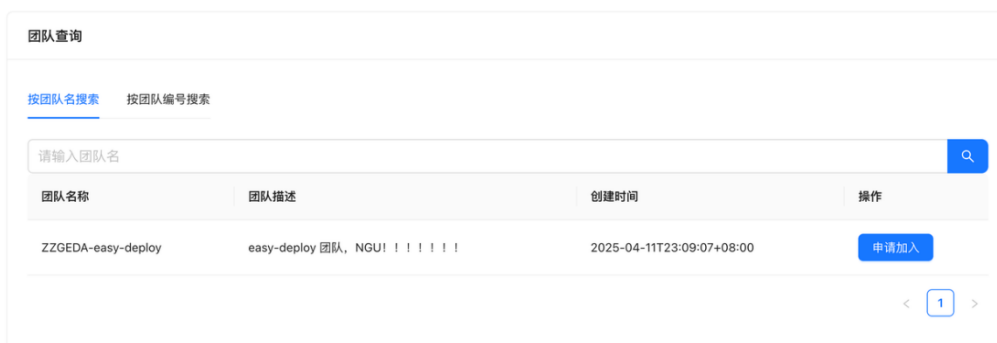


图 5.19 团队查询



图 5.20 团队申请加入邮件



图 5.21 团队入队审批

对于团队申请结果，无论团队管理员是“批准”还是“拒绝”申请者都将收到回复邮件以明确自己的请求状态，效果如图 5.22 团队审批结果邮件所示。



图 5.22 团队审批结果邮件

最终用户审批通过之后，系统将重新渲染“我的团队”模块，并将最新的团队成员信息渲染在页面上，并取分管理员身份和团队成员身份。效果如图 5. 23 团队简介-2 所示。



图 5. 23 团队简介-2

对于加入的团队成员如果在团队发展的后期需要离队的话则需要点击“申请离开”按钮进行离队流程。离队流程中团队管理员也将收到用户的申请请求邮件，并在“待审批请求”模块处对用户的离队请求进行审批。最终的审批结果也将向用户发送审批结果邮件以提示用户离队成功。待管理员审批结束后系统也将自动刷新“我的团队”模块，重新更新团队成员列表。

申请离队邮件如图 5. 24 离队申请邮件所示，审批结果的通知邮件如图 5. 25 离队审批结果邮件所示。



图 5. 24 离队申请邮件



图 5.25 离队审批结果邮件

5.2.9 代码仓库管理

代码仓库管理为用户提供了全 GitHub 代码仓库的全览功能。表示层通过用户绑定 GitHub Access token 调用 GitHub Open API 从而获取用户代码仓库的全部详细信息。

再获取 GitHub API 返回的代码仓库信息之后，依次渲染代码仓库列表、项目代码分支、文件树状目录、文件详细信息。

代码仓库管理功能效果图如图 5.26 代码仓库管理详情所示。

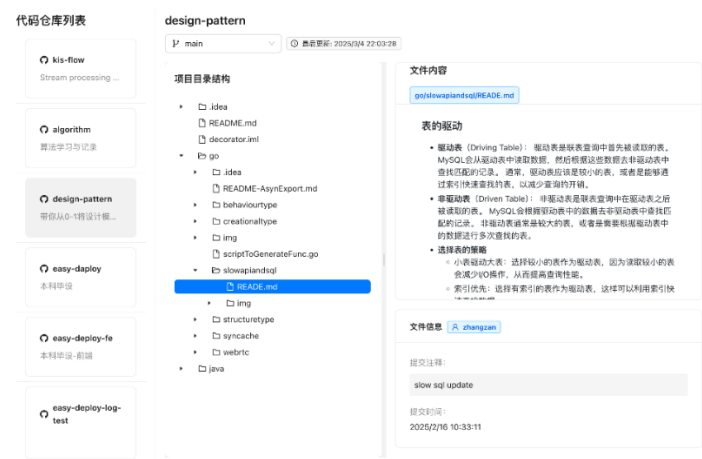


图 5.26 代码仓库管理详情

5.2.10 镜像文件管理

镜像文件 Dockerfile 管理是 Docker 镜像构建的前提。在“镜像管理”页面中，系统提供了 Dockerfile 镜像文件的保存、更新、删除、查看构建镜像构建历史功能。通过维护 Dockerfile 文件，用户可以极其方便的开始构建镜像并推送到镜像仓库中。此外对于 CI 部分，系统提供“shell 文件”绑定能力。用户需点击“查看 shell path”按钮查看用户自定义的 CI 构建脚本在代码仓库中的相对路径，从而在镜像构建时进行 CI 持续集成，自动完成项目的编译和测试工程，从而自动继续执行镜像构建和 CD 持续交付的全链路流程。

镜像文件管理页面如图 5.27 镜像文件管理所示。

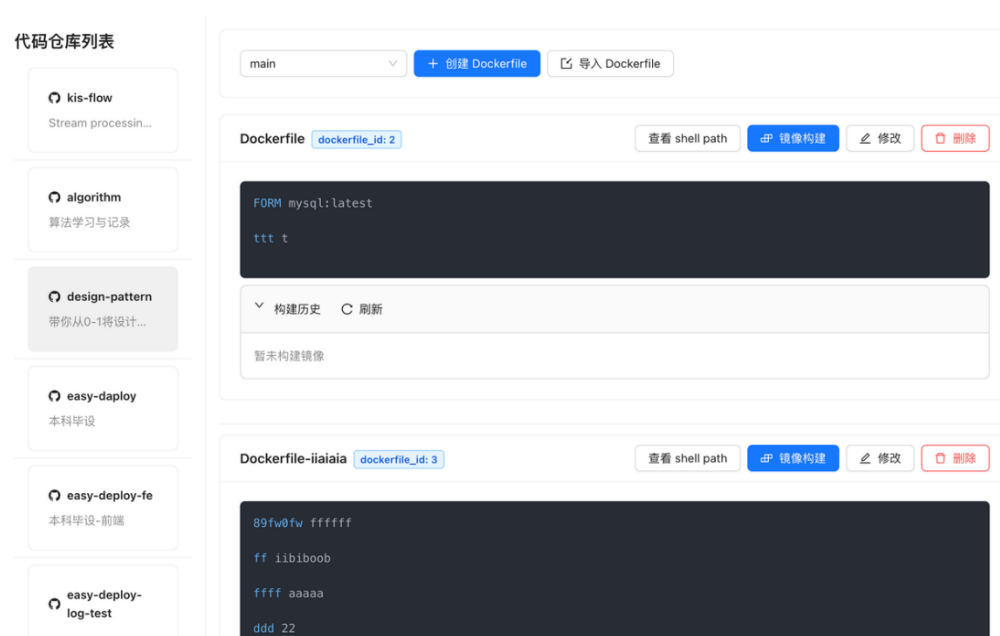


图 5.27 镜像文件管理

用户先在左侧的代码仓库列表处选择具体的代码仓库，然后再选择具体的分支之后便可以查看该代码仓库分支的所有 Dockerfile 文件。系统将以长列表的形式展示出每一个已创建的 Dockerfile 文件。

用户可以点击“创建 Dockerfile”或者“导入 Dockerfile”按钮实现项目 Dockerfile 的创建流程。

对于 Dockerfile 的创建，用户需要按照标准 Key-Value 的格式输入 Dockerfile 的关键字和对应执行参数。创建 Dockerfile 如图 5.28 创建 Dockerfile 所示。对

于关键字的使用，表示层提供了内置关键字快捷操作，支持用户快捷输入和快截选择，降低 Dockerfile 制作的难度。创建成功之后的 Dockerfile 文件将以 JSON Encode 的形式持久化存储在 MySQL 数据库中。

对于 Dockerfile 文件的更新,用户可以点击“修改”按钮对已创建的 Dockerfile 文件进行修改操作。同样用户可以点击“删除”按钮删除指定的 Dockerfile 文件。

图 5.28 创建 Dockerfile

系统支持 Dockerfile 的手动导入，效果如所示。用户需手动粘贴 Dockerfile 文件，表示层将自动解析用户在窗口中粘贴的 Dockerfile 文件，将其转换成标准 Key-Value 格式的 JSON 数据，然后交由后端进行持久化存储。效果如图 5.29 Dockerfile 文件导入所示。

图 5.29 Dockerfile 文件导入

此外用户可以点击 Dockerfile 文件下的“构建历史”隐藏栏以展示使用该 Dockerfile 文件构建出来的 Docker 镜像。镜像的构建历史将以倒序的形式展示已构建镜像的镜像全名、镜像名、创建者、构建时间。

由于同组内的团队成员已经同处于同一个 GitHub 代码仓库中，因此可以看见团队仓库内分支下的所有 Dockerfile 文件，以及每一个 Dockerfile 文件被不同团队成员的构建的构建历史。效果如图 5.30 Dockerfile 详情查看所示。



图 5.30 Dockerfile 详情查看

5.2.11 Docker 镜像构建

Docker 镜像构建是本系统的核心功能之一，其是 CI/CD 持续集成和持续交付功能开发的核心。功能使用页面如图 5.31 镜像构建-1 所示。



图 5.31 镜像构建-1

Docker 镜像构建分由 5 个部分组成，分别是“检查 Docker 账号”、“克隆代码仓库”、“生成 Dockerfile 文件”、“构建镜像”、“构建完成”5 个部分。

在“检查 Docker 账号”部分，表示层将通过网关层与应用层建立 WebSocket

长连接，并由应用层的“Docker 镜像流水线构建【CI】”模块进行 Docker 账号的环境检查、Docker 账号登录的初始化操作。初始化成功之后由用户通过点击“开始构建”按钮启动 Docker 镜像构建流程。

在第二部分“克隆代码仓库”中，表示层将通过 WebSocket 向“Docker 镜像流水线构建【CI】”模块发送流程启动请求。随后系统将通过“git clone”命令从 GitHub 代码仓库拉去最新的代码，以构建初始的 CI 运行环境。等待代码仓库克隆完成之后，系统将通过 WebSocket 发送成功指令，通知前端进入下一个流程。

接下来进入第三部分“生成 Dockerfile 文件”。由于 Dockerfile 文件是以 JSON 格式存储在 MySQL 中，所以需要将 JSON 数据格式转换为标准 Dockerfile 文件，以在“构建镜像”部分使用 Dockerfile 文件进行镜像构建。Dockerfile 文件将按照标准镜像构建范式输出到代码仓库中的根目录中。至此完成镜像构建的准备流程，效果如图 5.32 镜像构建-2 所示。



图 5.32 镜像构建-2

接下来进入第四部分“构建镜像”部分。这是 CI/CD 持续集成和持续交付的核心模块。在该部分之前，系统已经搭建好了镜像构架的基本环境，接下来就是执行 CI 流程的 shell 脚本，构建基础镜像，将镜像推送至 Docker 账号关联的远程镜像仓库。在“构建镜像”部分中，系统将进入代码仓库根目录，然后执行 shell 文件从而完成项目的编译、测试。随后通过输出的 Dockerfile 文件在项目根目录

中执行“docker build”命令。等待镜像构建成功，将使用“docker push”命令将已构建镜像推送至远程镜像仓库。至此实现项目的 CI/CD 持续集成和持续交付部分。

最后是进入第 5 部分“构建完成”。该部分是用于系统资源，防止系统长时间处于高负载状态。核心步骤是释放 WebSocket 连接。

在构建 Docker 镜像的过程中，系统将不断捕捉运行日志，并将运行日志通过 WebSocket 消息推送给表示层，以方便用户及时发现镜像构建过程中的问题，并及时修复，效果如图 5.33 镜像构建-3 所示。

```
#3 [internal] load .dockerignore #3 transferring context: 2B done #3 DONE 0.0s
#4 [1/1] FROM
docker.io/library/nginx:latest@sha256:5ed8fcc66f4ed123c1b2560ed708dc148755b6e4cbd8b943fab094f2c6bfa91e #4
resolve docker.io/library/nginx:latest@sha256:5ed8fcc66f4ed123c1b2560ed708dc148755b6e4cbd8b943fab094f2c6bfa91e
done #4 sha256:5ed8fcc66f4ed123c1b2560ed708dc148755b6e4cbd8b943fab094f2c6bfa91e 10.25kB / 10.25kB done #4
sha256:a70479423a2f7cfe7f3fca625ed0a2988947f0dd78f091ea89057b2855b4c2c6 2.29kB / 2.29kB done #4
sha256:de29066b274e2ed2dac558eef4fd46026fc03a8ff10f161d9909224984e689a9 0B / 40.75MB 0.1s #4
sha256:1f94323bafb2ac98d25b664b8c48b884a8db9db3d9c98921b3b8ade588b2e676 8.60kB / 8.60kB done #4
sha256:2cf157fc31fe8cc4501ab2599d6e2364defae458486140e6c3612dc321882b07 0B / 629B 0.1s #4
sha256:4509685639554482df00dc06f6ca412312afdce69f02c8d34b7482f309d5cf1 0B / 955B 0.1s
#4 sha256:2cf157fc31fe8cc4501ab2599d6e2364defae458486140e6c3612dc321882b07 629B / 629B 0.5s done #4
sha256:9b14c47aa231e326ee90e85d440f0007c902fd4b4c27d0d2f7a747dad7fe2939 0B / 404B 0.6s
#4 sha256:4509685639554482df00dc06f6ca412312afdce69f02c8d34b7482f309d5cf1 955B / 955B 0.9s done #4
sha256:fd8a9ced98467386fb8929b6d25a8ccd4d6e835370c456dc6e6d97155c98d1ca 0B / 1.21kB 1.0s
#4 sha256:9b14c47aa231e326ee90e85d440f0007c902fd4b4c27d0d2f7a747dad7fe2939 404B / 404B 2.4s done #4
sha256:c96c7b918bd5e8a6966d3e2b1d2ef2a284459902d33f7539fa3531cb13c7dcc7 0B / 1.40kB 2.5s
#4 sha256:de29066b274e2ed2dac558eef4fd46026fc03a8ff10f161d9909224984e689a9 2.10MB / 40.75MB 2.8s #4
sha256:fd8a9ced98467386fb8929b6d25a8ccd4d6e835370c456dc6e6d97155c98d1ca 1.21kB / 1.21kB 2.7s done
#4 sha256:de29066b274e2ed2dac558eef4fd46026fc03a8ff10f161d9909224984e689a9 5.24MB / 40.75MB 3.2s #4
```

图 5.33 镜像构建-3

5.2.12 Kubernetes 资源信息管理

Kubernetes 资源信息管理提供了 Kubernetes 资源配置文件管理功能，允许用户创建、更新、删除 Kubernetes 资源文件。页面效果渲染如图 5.34 Kubernetes 部署详情所示。

Kubernetes 资源配置文件与代码仓库绑定，用户可以查看从属于代码仓库的所有 Kubernetes 资源配置文件。因此可以查看团队内成员创建的所有配置文件，且所有配置文件按照升序分页的形式显示在窗口处。

在 Kubernetes 资源信息管理页面的最上层的“Docker 镜像构建历史”模块，允许用户查看其所选代码仓库构建的历史镜像，进而方便用户编写 Kubernetes 资源配置文件。



图 5.34 Kubernetes 部署详情

对于 Kubernetes 资源来说，其正常运行和容器的具体编排都离不开 Kubernetes 资源配置文件。通常来说，Kubernetes 资源配置文件以 YAML 文件格式存在，因此本系统重点针对 YAML 文件进行了特殊渲染，使得用户在表示层即可渲染出标准的 YAML 格式文件，提高了用户的使用体验。

在 Kubernetes 资源信息管理中，系统提供了两种 Kubernetes 资源文件的创建按钮，分别是“Deployment 配置构建”、“Service 配置构建”。

Deployment 是 Kubernetes 中用于管理无状态应用的工作负载资源对象。它负责创建和更新一组相同的 Pod 副本，确保指定数量的 Pod 实例始终处于运行状态。Service 是 Kubernetes 中用于暴露 Pod 服务的抽象层。它为一组 Pod 提供统一的访问入口，使得客户端可以通过一个固定的 IP 地址和端口号来访问这些 Pod，而无需关心具体的 Pod 实例。

用户点击“Deployment 配置构建”、“Service 配置构建”按钮完成配置文件的创建流程。创建弹窗展示如图 5.35 创建配置文件所示。



图 5.35 创建配置文件

由于 YAML 配置文件复杂，所以 YAML 文件均上传至 OSS，用户可以点击“OSS URL”展示列的“查看文件”文件从 OSS 直接获取文件并渲染。显示效果如图 5.36 OSS 查看文件所示。用户在查看文件的时候可以点击“编辑”按钮实现文件的编辑操作。



图 5.36 OSS 查看文件

在 Kubernetes 资源配置文件列表查看窗口处，用户可以点击“操作记录”按钮查看对应资源的操作情况和运行历史记录。该记录展示窗口以倒序分页的形式返回，分别展示资源的“创建”、“删除”、“检查”三种操作类型和“运行正常”、“运行停止”、“容器重启”三种状态。通过查看容器的运行日志，可以方便用户

直观的查看 Kubernetes 资源的运行状态。效果展示如图 5.37 Kubernetes 资源操作记录所示。操作记录展示容器的资源名称、命名空间、资源标签、执行命令等关键信息。

操作记录 ×

操作类型	状态	命名空间	资源名称	资源标签	执行命令	操作时间
检查	运行正常	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl get deployment nginx-deployment -n zzgeda-nginx	2025-04-05 11:09:52
检查	运行正常	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl get deployment nginx-deployment -n zzgeda-nginx	2025-04-05 11:09:50
检查	运行正常	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl get deployment nginx-deployment -n zzgeda-nginx	2025-04-05 11:08:30
检查	运行正常	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl get deployment nginx-deployment -n zzgeda-nginx	2025-04-05 11:07:30
检查	运行正常	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl get deployment nginx-deployment -n zzgeda-nginx	2025-04-05 10:45:09
检查	运行正常	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl get deployment nginx-deployment -n zzgeda-nginx	2025-04-05 10:42:32
创建	运行正常	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl apply -f k8s/7_nginx-deployment-3.yaml -n zzgeda-nginx	2025-04-05 09:55:59
删除	运行停止	zzgeda-nginx	nginx-deployment	{"app": "nginx"}	kubectl delete deployment nginx-deployment -n zzgeda-nginx	2025-04-04 17:29:13

共 1179 条记录 < 1 ... 114 115 116 117 **118** > 10 / page Go to Page 关闭

图 5.37 Kubernetes 资源操作记录

5.2.13 Kubernetes 资源操作

在“Kubernetes 部署”页面中，用户可以在表示层在经过网关层鉴权后与应用层的“Kubernetes 容器编排【CD】”模块进行 WebSocket 连接，从而操作 Kubernetes 客户端进行 0 代码地资源部署和信息查询。

用户在进行 WebSocket 绑定之后，可以点击“部署”或者“停止”按钮操作 Kubernetes 资源。应用层在收到表示层请求之后，会解析表示层请求参数，进而完成相应功能。其中对于“部署”功能，Kubernetes 客户端会从 OSS 获取 Kubernetes 资源配置文件，并在解析文件之后完成容器部署，从而实现 CD 持续部署流程。部署效果如图 5.38 Kubernetes 资源部署所示。

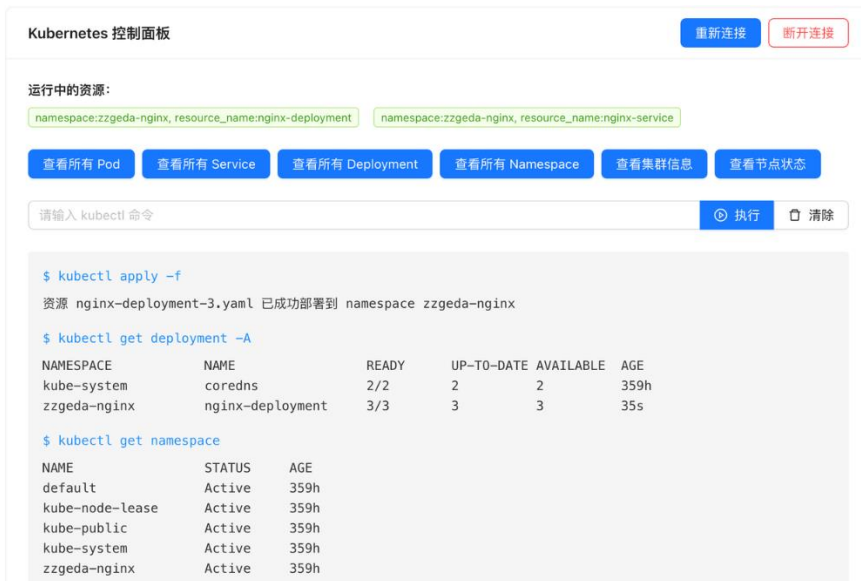


图 5.38 Kubernetes 资源部署

此外用户可以点击控制面板的预设按钮，进行相关的功能查询。预设按钮包括：“查看所有 Pod”、“查看所有 Service”、“查看所有 Deployment”、“查看所有 Namespace”、“查看集群信息”、“查看节点状态”。

效果展示如图 5.39 查看所有 Pod、图 5.40 查看所有 Service、图 5.41 查看所有 Deployment、图 5.42 查看所有 Namespace、图 5.43 查看集群信息、图 5.44 查看节点状态所示。

```
$ kubectl get pod -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-55cb58b774-77jtd	1/1	Running	6	359h
kube-system	coredns-55cb58b774-pkvhp	1/1	Running	6	359h
kube-system	etcd-docker-desktop	1/1	Running	6	359h
kube-system	fluentd-4766h	1/1	Running	3	149h
kube-system	kube-apiserver-docker-desktop	1/1	Running	6	359h
kube-system	kube-controller-manager-docker-desktop	1/1	Running	6	359h
kube-system	kube-proxy-748p6	1/1	Running	6	359h
kube-system	kube-scheduler-docker-desktop	1/1	Running	6	359h
kube-system	storage-provisioner	1/1	Running	12	359h
kube-system	vpnkit-controller	1/1	Running	6	359h
zzgeda-nginx	nginx-deployment-667c555644-h6ztk	1/1	Running	0	8m
zzgeda-nginx	nginx-deployment-667c555644-r7jfp	1/1	Running	0	8m
zzgeda-nginx	nginx-deployment-667c555644-v7r6z	1/1	Running	0	8m

图 5.39 查看所有 Pod

```
$ kubectl get svc -A
```

NAMESPACE	NAME	TYPE	CLUSTER-IP	PORT(S)
default	kubernetes	ClusterIP	10.96.0.1	443/TCP
kube-system	kube-dns	ClusterIP	10.96.0.10	53/UDP, 53/TCP, 9153/TCP
zzgeda-nginx	nginx-service	NodePort	10.98.208.77	80/TCP

图 5.40 查看所有 Service

```
$ kubectl get deployment -A
```

NAMESPACE	NAME	READY	UP-TO-DATE	AVAILABLE	AGE
kube-system	coredns	2/2	2	2	359h
zzgeda-nginx	nginx-deployment	3/3	3	3	8m

图 5.41 查看所有 Deployment

```
$ kubectl get namespace
```

NAME	STATUS	AGE
default	Active	359h
kube-node-lease	Active	359h
kube-public	Active	359h
kube-system	Active	359h
zzgeda-nginx	Active	359h

图 5.42 查看所有 Namespace

```
$ kubectl cluster-info
```

Kubernetes control plane is running at https://127.0.0.1:6443
Kubernetes version: v1.30.5

图 5.43 查看集群信息

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP
docker-desktop	Ready	master	359h	v1.30.5	192.168.65.3

图 5.44 查看节点状态

在 Kubernetes 控制面板处，用户可以查看团队创建的正在运行的资源。当鼠标悬浮在资源 Tab 处，将提示 Get 和 Describe 两个可点击按钮。这两个按钮可以直接查阅 Kubernetes 资源运行的具体信息。展示效果如图 5.45 运行中的资源、图 5.46 运行资源的操作、图 5.47 运行资源 Get 操作、图 5.48 运行资源

Describe 操作所示。



图 5.45 运行中的资源

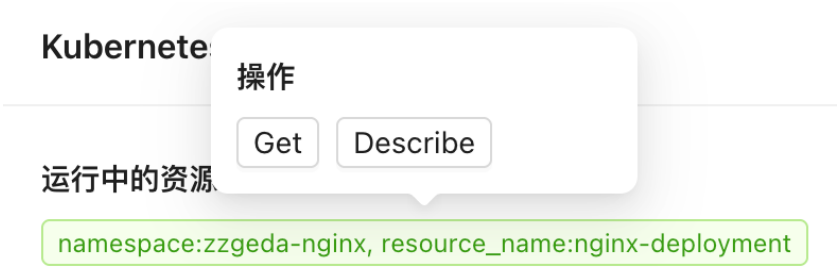


图 5.46 运行资源的操作

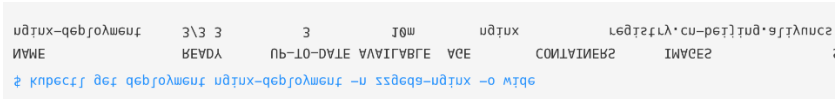


图 5.47 运行资源 Get 操作



图 5.48 运行资源 Describe 操作

通过这种方便快捷的模块化查询方式，使得用户可以在表示层快速掌握 Kubernetes 资源的运行状态，便于及时发现问题，快速解决。

5.2.14 EFK 日志搜集

本系统借用 EFK 技术，实现容器日志的无感搜集和日志的快速定位。在系统启动的时候会执行一个脚本用于在 Kubernetes 环境中运行 Fluentd 的 DaemonSet 容器，从而保证 Kubernetes 的每一个 Node 节点都能运行一个 Fluentd 容器，进而收集每一个 Node 节点上部署的容器的运行日志。

DemonSet 是一种工作负载资源对象，它的主要用途是保证在集群里的每个节点（或者符合特定规则的节点）上都运行一个 Pod 实例。Fluentd 是一个开源的数据收集器，可将各个节点上的日志收集起来并发送到 Elasticsearch 中。本系统按照容器名动态划分了不同的 Index，从而实现日志文件的分类存储，进而可以根据 Pod 的 容器名进行日志的快速划分。此外借由 Elasticsearch 特有的倒排索引结构，从而实现具体日志文件的快速定位。最终由 Kibana 提供页面友好的 Elasticsearch 数据查询客户端，进而通过模块化的按钮即可实现容器日志的快速搜索。

Index 划分如图 5. 49 Elasticsearch 动态索引所示，日志存储格式如图 5. 50 Elasticsearch 数据存储格式所示。

health	status	index
yellow	open	k8s_kube-controller-manager-2025.04.16
yellow	open	k8s_kube-proxy-2025.04.09
yellow	open	k8s_kube-scheduler-2025.04.13
yellow	open	k8s_kube-scheduler-2025.04.14
yellow	open	k8s_kube-controller-manager-2025.04.12
yellow	open	k8s_kube-controller-manager-2025.04.13
yellow	open	k8s_vpnkit-controller-2025.04.15
yellow	open	k8s_kube-controller-manager-2025.04.14
yellow	open	k8s_kube-controller-manager-2025.04.15
yellow	open	k8s_vpnkit-controller-2025.04.16
yellow	open	k8s_kube-controller-manager-2025.04.10
yellow	open	k8s_kube-controller-manager-2025.04.11
green	open	.kibana_task_manager_7.17.10_001
yellow	open	k8s_kube-scheduler-2025.04.11
yellow	open	k8s_kube-scheduler-2025.04.12
yellow	open	k8s_etcd-2025.04.11
yellow	open	k8s_etcd-2025.04.10
yellow	open	k8s_etcd-2025.04.13
yellow	open	k8s_etcd-2025.04.12

图 5. 49 Elasticsearch 动态索引

```

"hits": [
  {
    "_index": "k8s_nginx-2025.04.08",
    "_type": "_doc",
    "_id": "3ejdFJY80-t0Xv-j4Idd",
    "_score": 18.700205,
    "_source": {
      "log": "\"2025/04/08 10:05:22 [notice] 1#1: using the 'epoll' event method\"",
      "stream": "stderr",
      "docker": {
        "container_id": "b1570a5e881a38915de7660627dcfa362adef458563403d30a2df2e13c7624a2"
      },
      "kubernetes": {
        "container_name": "nginx",
        "namespace_name": "zzgeda-nginx",
        "pod_name": "nginx-deployment-667c555644-9f86b",
        "container_image": "registry.cn-beijing.aliyuncs.com/zzgeda/easy-deploy:1.1.4",
        "container_image_id": "docker-pullable://registry.cn-beijing.aliyuncs.com/zzgeda/easy-deploy@sha256:1e7a8076b624e4844ddc1b28a277d457dbce28a548da7f13a03dd6395adc3e60",
        "pod_id": "2e88baac-2fa4-45c3-ae28-80f0df700057",
        "pod_ip": "10.1.1.130",
        "host": "docker-desktop",
        "labels": {
          "app": "nginx",
          "pod-template-hash": "667c555644"
        },
        "master_url": "https://10.96.0.1:443/api",
        "namespace_id": "11bc1dd9-eaad-433e-9055-d712c50cfca9",
        "namespace_labels": {
          "kubernetes.io/metadata.name": "zzgeda-nginx"
        }
      }
    }
  }
]

```

图 5.50 Elasticsearch 数据存储格式

5.2.15 监控告警

在 EFK 的基础上，Flutend 组件将作为生产者同步向 Kafka 推送消息。本系统开发消费者从 Kafka 拉去容器日志消息，然后对日志消息进行日志等级过滤，对日志等于大于 Info 的日志进行报警。报警消息将推送给容器所属团队的管理员和容器的创建者。通过 Kafka 实现容器日志消息的异步存储，进而使用灵活性更高的消费者完成完整的容器日志监控告警功能。

告警邮件展示如图 5.51 容器监控告警所示，告警邮件展示容器所在的命名空间、Pod 名称、告警时间、告警级别、日志内容这几个关键信息。



图 5.51 容器监控告警

第6章 总结与展望

6.1 总结

本毕业设计成功构建了基于 Kubernetes 的一体化 DevOps 云平台，有效融合 Kubernetes 与 DevOps 理念，借助 Go 语言、TypeScript 语言、多种第三方中间件和第三方服务实现开发部署流程的标准化、高效化。

在技术实现上，系统采用 B/S 前后端分离的单体架构，前端利用 TypeScript 和 React 框架打造出交互友好的用户界面，后端通过 Go 语言和 Gin 框架提供稳定可靠的服务。同时，使用 EFK 实现 Kubernetes 的容器搜集；使用 Kafka 实现消息队列与监报告警；使用 Docker 和 Kubernetes 完成 DevOps CI/CD 持续集成、持续交付、持续集成的全流程；使用 OSS、Redis、MySQL 实现系统数据的高效存储，各组件协同工作，保障系统稳定运行。

功能层面，系统涵盖登录注册、个人信息管理、GitHub 账号管理、团队管理、代码仓库管理、镜像构建、Kubernetes 资源管理等核心功能，通过 CI/CD 流程实现从代码到部署的自动化，提升交付效率；EFK 日志搜集与监报告警功能，实时反馈系统状态，保障稳定性；数据库设计合理，支持各功能数据存储与交互。

通过本项目，不仅可以提升企业软件开发和部署效率，还为计算机相关专业学生提供实践参考，以白皮书形式开源在 GitHub 网站，助力知识传播与技术学习，推动 DevOps 和 Kubernetes 技术的教育普及。

6.2 展望

尽管本平台取得一定成果，但仍有提升空间，未来可从以下方面改进：

（1）性能优化：

随着用户量和业务规模增长，系统性能面临挑战。后续可优化 Kubernetes 资源调度算法，根据应用负载动态分配资源，提升集群资源利用率；对数据库进行索引优化、查询优化，提升数据读写性能；优化前端页面加载速度，采用代码压缩、缓存策略等技术，提高用户体验。

（2）功能拓展：

持续拓展功能，满足企业多样化需求。如集成更多第三方工具，支持更多云平台，增强系统兼容性与灵活性；引入人工智能和机器学习技术，实现

智能故障预测、自动修复等功能，提升系统智能化水平；完善权限管理体系，提供更细粒度权限控制，满足企业安全需求。

（3）安全强化：

容器化应用安全至关重要。后续加强容器安全防护，对运行容器进行定期检查，及时发现安全漏洞；完善数据加密机制，对敏感数据加密存储与传输，保护企业数据安全。

（4）易用性提升：

进一步简化系统操作流程，降低使用门槛。提供详细操作操作文档，帮助用户快速上手使用；优化用户界面设计，提升交互体验，使操作更便捷高效；加强系统自动化部署与配置功能，减少人工干预，提高部署效率。

通过以上改进，本平台将为企业数字化转型提供更完善、高效、安全的 DevOps 部署能力，在推动 DevOps 和 Kubernetes 技术发展道路上发挥更大作用。

参考文献

- [1] Ruparelia N B. Software development lifecycle models[J]. ACM SIGSOFT Software Engineering Notes, 2010, 35(3): 8-13.
- [2] 梁惠惠.对软件开发模式变迁的研究[J].现代信息科技,2019,3(22):1-4+8.DOI:10.19850/j.cnki.2096-4706.2019.22.001.
- [3] Burns B, Grant B, Oppenheimer D, et al. Borg, omega, and kubernetes[J]. Communications of the ACM, 2016, 59(5): 50-57
- [4] 翁涅元,单杏花,阎志远,等.基于Kubernetes的容器云平台设计与实践[J].铁路计算机应用, 2019,28(12):49-53.
- [5] 郭雷,毛玲燕.基于Kubernetes的企业级容器云平台设计与实践[J].信息技术与标准化,2022,(09):73-77.
- [6] CNCF.CNCF Cloud Native Survey China 2019[EB/OL]. (2020-10-13)[2025-04-19]. <https://www.cncf.io/blog/2020/10/13/cncf-cloud-native-survey-china-2019>.
- [7] CNCF.京东是怎样在镜像中央存储库用 Harbor 节省 60% 维护时间的[EB/OL]. (2020-06-24)[2025-04-19]. <https://www.cncf.io/case-studies-cn/jdcom-harbor>.
- [8] CNCF.中国民生银行借助 Kubernetes，实现了旧有应用程序转型，将其转向人工智能、区块链和大数据[EB/OL]. (2020-01-23)[2025-04-19]. <https://www.cncf.io/case-studies-cn/cmbc>.
- [9] Kubernetes.蚂蚁金服案例研究 | Kubernetes (K8s) 容器编排系统[EB/OL]. (2023-05-20)[2025-04-19]. <https://v1-30.kubernetes.ac.cn/case-studies/ant-financial>.
- [10] 牛晓玲,吴蕾.DevOps发展现状研究[J].电信网技术,2017,(10):48-51.
- [11] 杜军.基于Kubernetes的云端资源调度器改进[D].浙江大学,2016.
- [12] 单朋荣,杨美红,赵志刚,等.基于Kubernetes云平台的弹性伸缩方案设计与实现[J].计算机工程,2021,47(01):312-320.
- [13] 杨茂,陈莉君.基于Kubernetes的容器自动伸缩技术的研究[J].计算机与数字工程,2019,47(09):2217-2220+2232.
- [14] 胡鹏,常朝稳,祝现威,等.面向持续集成的回归测试优化方法[J].计算机应用研究,2021,38(12):3709-3714.
- [15] Tenable Cloud Risk Report 2024[EB].<https://www.tenable.com/cyber-exposure/tenable-cloud-risk-report-2024>.

- [16] CNCF. Project Journey Report[EB/OL]. (2023-06-08)[2025-04-19]. <https://www.cncf.io/reports/kubernetes-project-journey-report/>.
- [17] IBM. 什么是Kubernetes? [EB/OL]. (2024.03.11)[2025-04-19]. <https://www.ibm.com/cn-zh/topics/kubernetes>.
- [18] Debian.Spinnaker 介绍 – Netflix 的持续交付平台[EB/OL]. (2023-05-20)[2025-04-19]. <https://debian.cn/articles/448>
- [19] Pulumi. What is Pulumi? | Pulumi Docs[EB/OL]. (2023-05-20)[2025-04-19]. <https://www.pulumi.com/docs/iac/concepts>.
- [20] RubakAdam, TaheriJavid. Machine Learning for Predictive Resource Scaling of Microservices on Kubernetes Platforms. //Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing. New York, NY, USA: Association for Computing Machinery, 2024.
- [21] Jorge-Martinez D, Butt S A, Onyema E M, et al. Artificial intelligence-based Kubernetes container for scheduling nodes of energy composition[J]. International Journal of System Assurance Engineering and Management, 2021: 1-9.
- [22] LeiteLeonardo, RochaCarla, KonFabio, MilojicicDejan, MeirellesPaulo. A Survey of DevOps Concepts and Challenges. ACM Comput. Surv., 52(6), 2019.
- [23] 丁海斌,崔隽,陆凯.基于Docker的DevOps系统设计与实现[J].指挥信息系统与技术,2017,8(03):87-92.
- [24] 陈博,周亦敏.基于Kubernetes的CI/CD平台[J].计算机系统应用,2020,29(12):268-271.

致 谢

成长至今，我始终相信天道酬勤，并最终依靠自己的双手实现了我本科阶段的梦想。回望来时路，满路荆棘险阻。在此我特别想感谢我的家人对我 22 年来无微不至的关怀，是我的家人为我提供了良好的成长环境和家庭氛围。正是因为我的家人才有了如今的我。

但是人生的旅程才刚刚开始，未来的道路还很漫长。我将继续披荆斩棘，勇往直前。就像乔布斯先生说的那样——“找到你热爱的事情”。在人生旅程上，我将一直追逐我所热爱之事。

感谢来时路和那个永不放弃的少年。